版本：

軟體詳細設計說明書

摘要				
發 文 單 位		副總經理	主管	撰寫人
發 文 日 期				

Format ver. 2.0

修改歷史

修改完稿日期	修改內容說明	作者	參與者

目錄

1 引言 6

1.1 目的 6
1.2 實驗平台 6
1.3 關鍵名詞說明 6

2 IMG_PROC 6

2.1 使用 OPENCV 影像讀取與寫出的基本操作 6
2.1.1 new_image_2d 7
2.1.2 new_image_3d 8
2.1.3 load_image_2d 8
2.1.4 load_image_3d 9
2.1.5 save_image_2d 9
2.1.6 save_image_3d 10
2.1.7 save_image_1d_to_2d 10

3 EDGE_DETECTION 11

3.1 軟體架構 11
3.2 軟體流程 12
3.3 資料結構 12
3.3.1 Contours 12
3.4 指令功能說明 13
3.4.1 canny2 13
3.4.2 canny 13
3.4.3 gauss_conv 15
3.4.4 grad 16
3.4.5 boundary_neighborhood_9n 16
3.4.6 findK 17
3.4.7 partition 18
3.4.8 fill_imap 18
3.4.9 clean 19
3.4.10 fill_edges3c 20

4 IMPROVED_HOUGH 21

4.1 軟體架構 21
4.2 軟體流程 22
4.3 資料結構 22
4.3.1 Point2D 22
4.3.2 image_primitives 23

4.3.3 line_Obj 23
4.3.4 line_Points 23
4.3.5 lens_distortion_model 23
4.3.6 image_XLD 24
4.4 IMAGE_PRIMITIVES 相關函式 24
主要函式 24
4.4.1 Build_Image_Primitives 24
4.4.2 pass_line_Obj 25
4.4.3 pass_image_primitives 25
4.5 指令功能說明 26
4.5.1 line_equation_distortion_extraction_improved_hough 26
4.5.2 update_contours 28
4.5.3 evaluation 29
4.5.4 polynomial_evaluation 30
4.5.5 orientation_update 30
4.5.6 update_rsqmax 31
4.5.7 norm 32
4.5.8 norm2 32
4.5.9 substract 33
4.5.10 voting 33
4.5.11 maximum_selection 34
4.5.12 capturing_points 36
4.5.13 Point2Line_distance 37
4.5.14 line_point_push_back 38
4.5.15 ensure_consistent_line_orientations 38
4.5.16 compute_orientation_sign 39
4.5.17 erase_point 40
4.5.18 recompute_line_equations 41
4.5.19 points_to_equation 42
4.5.20 remove_and_join 43
4.5.21 erase_line 44
4.5.22 Point2Point_distance_min 45
4.5.23 combine_line 45
5 PARAMETER_OPTIMIZATION 46
5.1 軟體架構 46
5.2 軟體流程 47
5.3 資料結構 47
5.3.1 Point2D 47
5.3.2 image_primitives 48

5.3.3 line_Obj *48*
5.3.4 line_Points *48*
5.3.5 lens_distortion_model *48*
5.3.6 image_XLD *49*
5.3.7 E_accumulators *49*
5.4 IMAGE_PRIMITIVES 相關函式 50
主要函式 50
5.4.1 Build_Image_Primitives *50*
5.4.2 pass_line_Obj *50*
5.4.3 pass_image_primitives *51*
5.5 指令功能說明 51
5.5.1 iterative_optimization *51*
5.5.2 total_points *53*
5.5.3 energy_minimization *54*
5.5.4 model_center_estimation_2p *55*
5.5.5 update_rsqmax *57*
5.5.6 E_accumulators_reset *58*
5.5.7 compute_accumulators *58*
5.5.8 update_den_ks *59*
5.5.9 update_ldm_and_increment_accumulator *60*
5.5.10 distortion_points_to_line_equation *61*
5.5.11 compute_gradient *62*
5.5.12 compute_hessian *63*
5.5.13 ami2_gauss *63*
5.5.14 evaluation *64*
5.5.15 polynomial_evaluation *65*
5.5.16 check_invertibility *65*
5.5.17 line_equation_distortion_extraction_improved_hough *67*
6 IMAGE_CALIBRATION 67
6.1 軟體架構 67
6.2 軟體流程 68
6.3 資料結構 68
6.3.1 Point2D *68*
6.3.2 lens_distortion_model *69*
6.4 指令功能說明 69
6.4.1 undistort_image_inverse *69*
6.4.2 update_rsqmax *70*
6.4.3 build_l1r_vector *71*
6.4.4 polynomial_evaluation *71*

6.4.5 compute_division_l1r *72*
6.4.6 compute_scale *73*
6.4.7 evaluation *75*
6.4.8 multiply *76*
6.4.9 substract *77*
6.4.10 norm *77*
6.4.11 find_point *78*

7 參考文獻 79

1 引言

本說明書將會詳細介紹本計畫中所使用的軟體，包含其目標、實驗平台、軟體流程架構、資料結構以及每一個指令的功能介紹。

1.1 目的

本計畫的目標為以 C 語言實現 HALCON 中的三個函式，分別為 radial_distortion_self_calibration() 、 change_radial_distortion_cam_par () 以及 change_radial_distortion_image ()，實作方法為蒐集近期與畸變影像校正相關的文獻與書籍，並重新實現以及改良其演算法，最後與 HALCON 比較結果。

1.2 實驗平台

(1) 系統環境： PC–Windows 10–64 位元，使用一個執行緒（CPU：Intel i5-8400H），記憶體:32GB

(2) 開發平台： C

(3) 開發環境： Visual studio 2015

1.3 關鍵名詞說明

(1) 像素點(Pixel point)：單位為整數型的座標點

(2) 影像 (uchar**)：單通道的灰階影像

(3) 影像 (uchar***)：三通道的彩色影像

(4) 邊緣 (line_Obj)：由多個像素點組成的邊緣

(5) 校正參數集合 (image_primitives)：由多個邊緣(line_Obj)和畸變模型組成的集合

2 Img_proc

2.1 使用 Opencv 影像讀取與寫出的基本操作

下表為本專案中影像讀取與寫出與其他影像處理的基本操作指令架構圖，

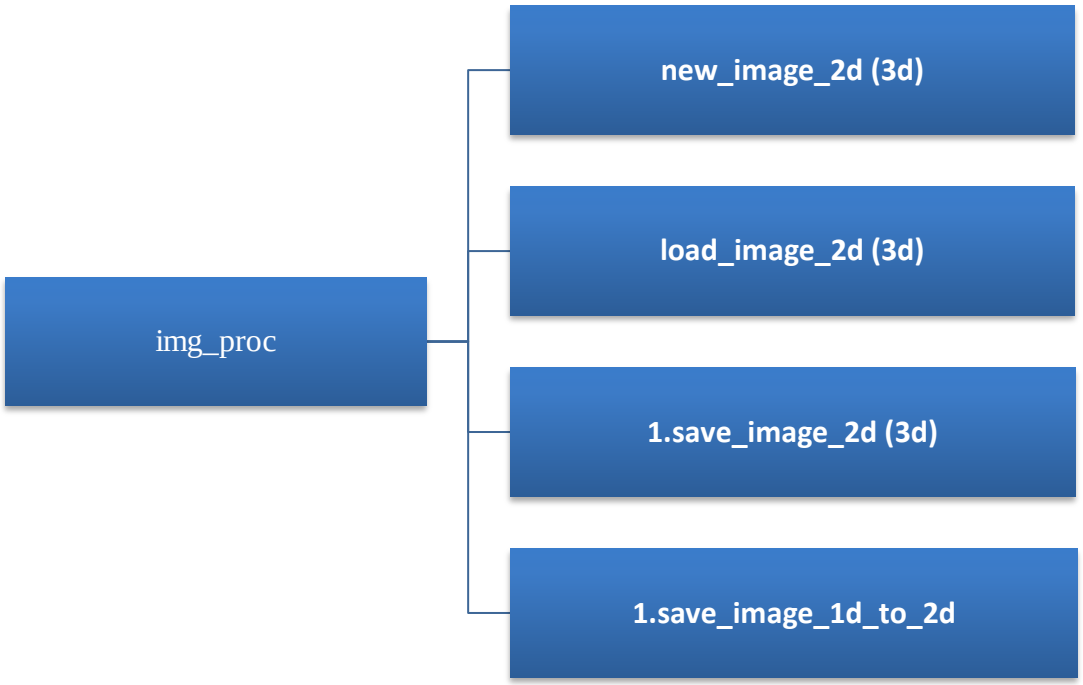

圖 1、img_proc 基本影像處理的指令架構圖

2.1.1 new_image_2d

uchar** new_image_2d(int r, int c, int ch)

功能描述

根據輸入參數的長寬資訊宣告一 2 維灰階矩陣。

Input

- r (int)：影像的高。
- c (int)：影像的寬。
- ch (int)：影像的頻道數 (因 2 維灰階影像，此參數應保持為 1)。

Output

無

Return

- img (uchar **)：回傳一 2 維灰階矩陣，entry 型態為 unsigned integer 8。

設計方法

依據輸入的長寬資訊動態宣告一 2 維陣列，並初始化每個 entry 為 0。

使用方法

若想輸出灰階或黑白影像時，可先根據需要的長寬宣告一 2 維陣列進行矩陣運算。

2.1.2 new_image_3d

uchar** new_image_3d(int r, int c, int ch)

功能描述

根據輸入參數的長寬資訊宣告一 3 維 RGB 矩陣。

Input

- r (int) : 影像的高。
- c (int) : 影像的寬。
- ch (int) : 影像的頻道數 (因 3 維 RGB 影像，此參數應保持為 3)。

Output

無

Return

- img (uchar **) : 回傳一 3 維灰階矩陣，entry 型態為 unsigned integer 8。

設計方法

依據輸入的長寬資訊動態宣告一 3 維陣列，並初始化每個 entry 為 0。

使用方法

若想輸出 RGB 影像時，可先根據需要的長寬宣告一 3 維陣列進行矩陣運算。

2.1.3 load_image_2d

uchar** load_image_2d(char* filename, int* shape)

功能描述

讀取影像並回傳 2 維灰階影像。

Input

- filename (char *) : 來源影像
- shape (int*) : 影像維度大小 (高/寬/頻道數)。

Output

- shape (int*) : 影像維度大小 (高/寬/頻道數)。

Return

- img (uchar **) : 回傳一 2 維灰階影像。

設計方法

使用 opencv 中 cvLoadImage 函數來讀取影像，並複製像素資訊至已宣告的 2 維陣列並回傳。

使用方法

讀取灰階影像至 2 維陣列以利後續的矩陣運算操作。

2.1.4 load_image_3d

uchar*** load_image_3d(char* filename, int* shape)

功能描述

讀取影像並回傳 3 維 RGB 影像。

Input

- filename (char *)：來源影像。
- shape (int*)：影像維度大小 (高/寬/頻道數)。

Output

- shape (int*)：影像維度大小 (高/寬/頻道數)。

Return

- img (uchar **)：回傳一 3 維 RGB 影像。

設計方法

使用 opencv 中 cvLoadImage 函數來讀取影像，並複製像素資訊至已宣告的 3 維陣列並回傳。

使用方法

讀取 RGB 影像至 3 維陣列以利後續的矩陣運算操作。

2.1.5 save_image_2d

void save_image_2d(char* img_name, uchar** img, int* shape)

功能描述

將一 2 維灰階陣列的資訊儲存成影像輸出。

Input

- img_name (char *)：寫出的影像名稱。
- img (uchar **)：欲寫出的 2 維灰階影像資訊。
- shape (int*)：影像維度大小 (高/寬/頻道數)。

Output

無

Return

無

設計方法

使用 opencv 中的 cvSaveImage 函式將一 2 維灰階陣列的資訊儲存成影像輸出。

使用方法

將操作後的 2 維陣列資訊儲存成影像檔寫出。

2.1.6 save_image_3d

void save_image_3d(char* img_name, uchar*** img, int* shape)

功能描述

將一 3 維陣列的資訊儲存成影像輸出。

Input

- img_name (char *) : 寫出的影像名稱。
- img (uchar **) : 欲寫出的 3 維 RGB 影像資訊。
- shape (int*) : 影像維度大小 (高/寬/頻道數)。

Output

無

Return

無

設計方法

使用 opencv 中的 cvSaveImage 函式將一 3 維陣列的資訊儲存成影像輸出。

使用方法

將操作後的 3 維陣列資訊儲存成影像檔寫出。

2.1.7 save_image_1d_to_2d

void save_image_1d_to_2d(char* img_name, uchar* img, int* shape)

功能描述

將 1 維陣列的資訊轉成指定的 2 維陣列大小後並儲存成影像輸出。

Input

- img_name (char *)：寫出的影像名稱。
- img (uchar*) : 1 維陣列的影像資訊 (row-majored)。
- shape (int*)：影像維度大小 (高/寬/頻道數)。

Output

無

Return

無

設計方法

依據指定的 2 維陣列大小將 1 維的影像資訊轉成 2 維陣列後再呼叫 save_image_2d 函式將轉成 2 維陣列的影像資訊儲存成影像輸出。

使用方法

當影像存成 1 維陣列的形式時，可用此函數將 1 維影像資訊輸出成影像檔。

3 Edge_detection

3.1 軟體架構

如下圖所示，本函式主要對輸入影像求取輪廓資訊。

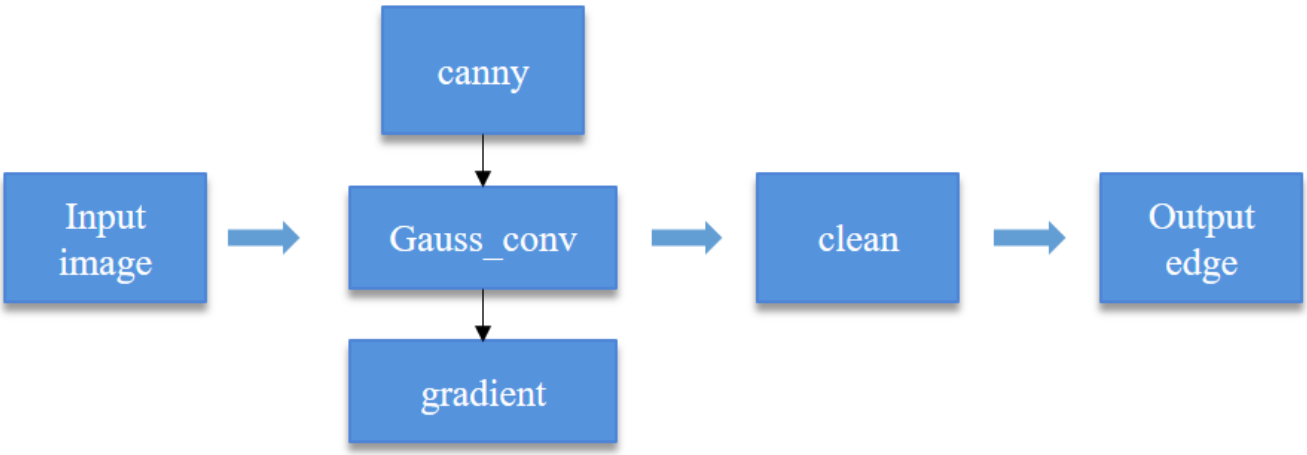

圖 2、Edge_detection 架構圖

3.2 軟體流程

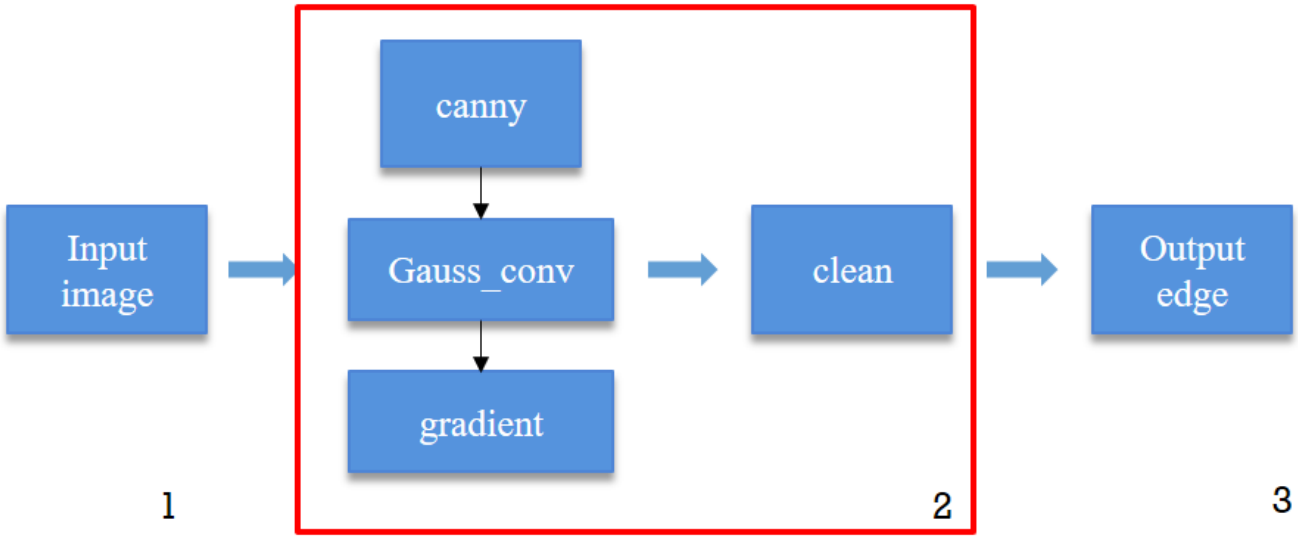

圖 3、Edge_detection 流程圖

1. 讀入影像
2. 利用 canny 偵測輪廓
3. 回傳檢測到的輪廓資訊

3.3 資料結構

功能描述

基本資料型態定義

資料結構

```
typedef char                I8;
typedef unsigned char       U8;
typedef short               I16;
typedef unsigned short      U16;
typedef int                 I32;
typedef unsigned int        U32;
typedef long long           I64;
typedef unsigned long long  U64;
typedef float               F32;
typedef double              F64;
```

3.3.1 Contours

功能描述

儲存輪廓點之資訊

資料結構

```
struct contours {
	bool* get_c;
	float* get_x;
	float* get_y;
	float* get_sine;
	float* get_cosine;
	int* index;
	int index_size;
};
```

3.4 指令功能說明

3.4.1 canny2

struct contours contours = canny2(gray, edges, shape, canny_low_threshold, canny_high_threshold);

功能描述

對輸入影像利用 canny edge detector 求出輪廓。

Input

- gray (uchar*)：來源灰階影像
- edges (uchar*)：用來存放計算後的輪廓影像
- shape (int*)：儲存輸入影像之 width、height、通道數
- canny_low_threshold(float)：計算 low_threshold 時需用到之參數
- canny_high_threshold(float)：計算 high _threshold 時需用到之參數

Output

無

Return

contours：計算出的輪廓

設計方法

此函式針對要輸出之輪廓變數作 memory allocate 以及初始化，利用函式 canny 求出輪廓點資訊，並把輪廓點資訊存入要回傳的變數 contours 裡。

使用方法

輸入欲得到輪廓之影像及計算 canny threshold 的參數。

3.4.2 canny

void canny(uchar* input, uchar* output, int* shape, float* sine, float* cosine, int* x, int* y, float per_low,

float per_high);

功能描述

實際包含 canny 函式的 function，對輸入影像做 canny edge detection。

Input

- input(uchar*)：輸入影像
- output(uchar*)：用來存放計算後的輪廓影像
- shape(int*)：儲存輸入影像之 width、height、通道數
- per_low(float)：計算 low_threshold 時需用到之參數。
- per_high(float)：計算 high _threshold 時需用到之參數。

Output

- sine： 輪廓中的 sine。
- cosine： 輪廓中的 cosine。
- x_pos： 輪廓中的 x。
- y_pos： 輪廓中的 y。

Return

無

設計方法

先使用高斯濾波器(3.4.3 gauss_conv)對影像做去雜訊的動作，接著分別找出 x 與 y 方向的梯度，再使用 low threshold 和 high threshold 對整張圖片的 gradient 做分類，將每個 pixel 分為邊緣或非邊緣。

若此梯度強度大於 high threshold，則此點被分類為 edge，若 pixel 的梯度強度介於 low threshold 以及 high threshold 之間，確認此點的八鄰域點是否有被分類為 edge 的點，若有則此點被歸為 edge，再根據此點判斷新的邊緣點，整合邊緣。

接著會進行 clean edge，將不必要的 edge 加以清除，以上的所有功能將會以一個個函式呈現，並可以在接下來的小節找到詳細的說明。

使用方法

此函式是利用以指標型式讀入之 input 圖片，經過 canny edge detection 運算後，將得到的 x_pos、y_pos、sine、cosine 的值放入事先 malloc 好並且同為此函式輸入的指標陣列中，以利後續使用這些指標陣列來表示圖片的輪廓。

3.4.3 gauss_conv

void gauss_conv(uchar* img, float* img_conv, int* shape, double sigma_x, double sigma_y, double precision);

功能描述

使用輸入參數對應的高斯遮罩對輸入之圖片進行去除雜訊的動作。

Input

- img(uchar*)：輸入圖片
- shape(int*)：儲存輸入影像之 width、height、通道數
- sigma_x(double)：高斯遮罩公式中的 sigma 值
- sigma_y(double)：高斯遮罩公式中的 sigma 值
- precision(double)：高斯遮罩公式中的 precision 參數

Output

- img_conv(float*)：預先 malloc 好的二維指標陣列，為高斯模糊後的圖片。

Return

無

設計方法

程式實作方面使用高斯濾波近似法[9]分別對圖像的 x 方向與 y 方向做一維的卷積濾波。近似方法利用熱方程式的解來趨近高斯濾波的結果。

以 x 方向軸為例，先算出熱方程式中的解L_x,其中σ為σ_x與σ_y之最大值:

$$\lambda_x = \sigma_x * \sigma_x/(2 * precision * \sigma)$$

$$L_x = \frac{1 + 2\lambda_x - \sqrt{4\lambda_x + 1}}{2\lambda_x}$$

再經由三次迴圈進行迭代:

$$U_i = I_i + L_x * U_{i-1}$$

$$I_i = U_i + L_x * I_{i+1}$$

$$I_i = I_i * (L_x/\lambda_x)$$

其中U_i對應程式中的 step[n]、I_i為 img_conv[x]、l_y 為L_x,以及 v_y 為L_x/λ_x。

(為使程式的命名與參考文獻[9]的公式作對應，底標的 i 為程式中 for 迴圈的變數。以及存在程式中 y 軸與 x 軸所對應 row 和 column 上可能會造成變數 x 與 y 互相對調的問題。)

使用方法

在使用 edge detector 找出影像中的 edges 前，通常會先對影像作高斯濾波以降低雜訊，使邊緣運算子所找出的邊界較為單純。

3.4.4 gradient

void gradient(float* img, int* shape, float* grad_x, float* grad_y);

功能描述

計算輸入圖片的 gradient。

Input

- img(float*)：輸入圖片
- shape(int*)：儲存輸入影像之 width、height、通道數

Output

- grad_x(float*)：以指標陣列型態存放計算後的 x 方向 gradient。
- grad_y(float*)：以指標陣列型態存放計算後的 y 方向 gradient。

Return

無

設計方法

使用下列遮罩掃過每一個像素點，分別計算每個像素的 x 方向與 y 方向梯度。

$$\frac{\partial I}{\partial x} = 0.5\begin{bmatrix} -\frac{2-\sqrt{2}}{2} & 0 & \frac{2-\sqrt{2}}{2} \\ -(\sqrt{2}-1) & 0 & \sqrt{2}-1 \\ -\frac{2-\sqrt{2}}{2} & 0 & \frac{2-\sqrt{2}}{2} \end{bmatrix} \quad and \quad \frac{\partial I}{\partial y} = 0.5\begin{bmatrix} -\frac{2-\sqrt{2}}{2} & -(\sqrt{2}-1) & -\frac{2-\sqrt{2}}{2} \\ 0 & 0 & 0 \\ \frac{2-\sqrt{2}}{2} & \sqrt{2}-1 & \frac{2-\sqrt{2}}{2} \end{bmatrix}$$

使用方法

輸入一張圖片，並且輸入兩個個預先 allocate 好的二維指標陣列，一個用來儲存 x 方向梯度，一個用來儲存 y 方向梯度，函式執行完後 x 與 y 的兩個指標陣列便會存有梯度資訊。

3.4.5 boundary_neighborhood_9n

int** boundary_neighborhood_9n(int width, int height);

功能描述

在使用 grad 這個函式計算梯度時，需要使用遮罩掃過所有像素點，故需要此函式來補足計

算邊界點時梯度所需的資訊。

Input

- width(int) ：輸入圖片的寬度，為 column 的數量。
- height(int) ：輸入圖片的高度，為 row 的數量。

Output

無

Return

- b(int**)：存有邊界梯度資訊的二維陣列

設計方法

以 index 形式表示邊界資訊，分為 first corner、boundary horizonal line、second coner、boundary vertical line 分別將邊界資訊填入回傳的二維指標陣列中。

使用方法

由於是以 index 形式表示，故只需要輸入影像的長寬，便可找出邊界像素對應的 index。

3.4.6 findK

float findK(float* array, int left, int right, int k)

功能描述

從一個給定的陣列中，以遞迴的方式找出第 k 大的數值。其優點為不須經過排序複雜的過程，可以大量減少計算時間。

Input

- array(float*)：欲找出第 k 大的數值的陣列
- left(int)：陣列左邊的 index
- right(int)：陣列右邊的 index
- k(int)：希望找出的第 k 個值 index

Output

無

Return

int : 回傳找到的第 k 大數值。

設計方法

不斷的比較與執行 partition 這個函式，直到目前分群的基準點為 k 便停止。

使用方法

輸入欲尋找的矩陣，與矩陣左右兩側的 index，與想找出第幾(k)大的數值，便會回傳整個矩陣中第 k 大的值。

3.4.7 partition

float partition(float* array, int left, int right)

功能描述

findK 函式中使用到尋找第 k 大的子函式。目的為將目前陣列分為比某數大與比某數小。

Input

- array(float*)：欲找尋並且在找尋過程中的陣列
- left(int) ：陣列左邊的 index
- right(int) ：陣列右邊的 index

Output

無

Return

int：目前分群的基準點 index

設計方法

初始時會以最左邊的數值為基準點，將整個陣列分群，接著回傳分群基準點的 index，主函式 findK 便會以當下基準點 index 和 k 比較來更新目前基準點。

使用方法

輸入整個當下矩陣與矩陣的左右 index，便可得當下分群的基準 index。

3.4.8 fill_imap

void fill_imap(int* imap, int* shape, float* gradien_norm, double low_threshold, int k)

功能描述

對於梯度介於 low threshold 與 high threshold 之間的像素點決定是否需要轉變為邊緣點。

Input

- imap(int*)：為先前演算法算出是否為邊緣點的指標陣列
- shape(int*)：儲存輸入影像之 width、height、通道數

- gradien_norm(float*)：儲存梯度資訊的指標陣列
- low_threshold(double) ：先前演算法找出的 low threshold
- k(int) ：目前正在進行判斷的像素點

Output

- imap(int*)：此時的 imap 為修改過後的 imap

Return

無

設計方法

將以當前像素點為中心的八個鄰近點以遞迴的方式不斷重複 fill_imap 這個函式進行邊緣點的更新。

使用方法

輸入當前 imap、整張圖的梯度資訊、low threshold 與目前像素點的 index，便可以得到更新過後的 imap。

3.4.9 clean

```
struct contours clean(
    struct contours contours,
    int* shape,
    int neighborhood_radius,
    int min_neighbor_points,
    double min_orientation_value,
    int min_distance_point);
```

功能描述

對當下的邊緣點進行清除的動作，以除去多餘的邊緣點。

Input

- contours(struct contours) ：欲進行清除動作的 contours
- shape(int*)：儲存輸入影像之 width、height、通道數
- neighborhood_radius(int) ：為一個超參數，代表定義的鄰居點半徑
- min_neighbor_points(int) ：為一個超參數，代表最小鄰居的數量
- min_orientation_value(double) ：為一個超參數，代表鄰居點方向最小平均的 scalar product

- min_distance_point(int) ：為一個超參數，代表 contours 點之間的最小距離

Output

- contours(struct contours) ：以進行清除過後的 contours

Return

無

設計方法

此部分將針對較不必要的輪廓點加以刪除以減少之後計算時間。首先，每個輪廓點會先計算自己有幾個 neighborhood points，此演算法預設以當前點為中心，5*5 的 window 內皆屬於 neighborhood points，並將每個 neighborhood points 與自己的 orientation 作內積，所有內積值累加存為此點的 scalar_product。若滿足以下兩個條件其中一個，則此點便會被清除掉。

條件一：此點的 neighborhood points 個數小於 2

條件二：此點的 scalar_product 小於此點的 neighborhood points 個數乘以 0.95 上述清除步驟會重複四次，剩下的輪廓點會再進行下一個判斷

條件三：以 5*5 的 window 掃整張影像，確保在每個 window 內最多只有一個輪廓點，若 window 內有複數個輪廓點，則保留 scalar_product 最大的點，其餘的清除。window 步伐為 window 大小，因此不會有重疊的問題。

清除前後的輪廓影像如下圖：

圖 4、清除前後的輪廓影像

使用方法

輸入當前的輪廓格式與 neighborhood_radius、min_neighbor_points、min_orientation_value、min_distance_point 這些必要的超參數，便會得到清除不必要輪廓後的結果。

3.4.10 fill_edges3c

void fill_edges3c(struct contours contours, int* shape, uchar*** edges3c, char* image_name)

功能描述

將計算過程中以平面表示的像素點轉換為真實圖片三維的形式。

Input

- contours(struct contours) ：清除過後乾淨的輪廓點
- shape(int*)：儲存輸入影像之 width、height、通道數
- image_name(char*)：欲儲存的輪廓檔案名稱

Output

- edges3c(uchar***)：轉變為三維後的邊緣圖片

Return

無

設計方法

依據一維圖片的排列規則，將圖片資料轉換為三維。

使用方法

輸入一維的輪廓資料與預先 malloc 好的三維空間，便可以得到一個三維正常圖片空間的輸出。

4 Improved_hough

4.1 軟體架構

如下圖所示，本函式主要進行直線檢測並算出畸變參數 k1，以及對校正後的輪廓進行整併。輸入輪廓先經過畸變模型得到校正後的輪廓，利用校正後的輪廓作霍夫變換檢測直線，同時不斷更新畸變參數 k1，經過數次迴圈後得到最佳的畸變參數 k1，針對最佳畸變模型校正出的輪廓進行整理合併，輸出畸變模型及輪廓。

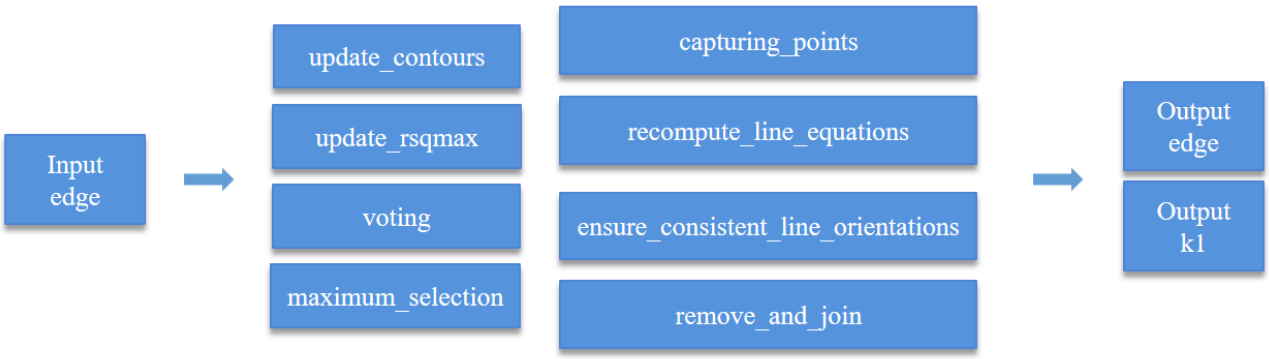

圖 5、Improved_hough 架構圖

4.2 軟體流程

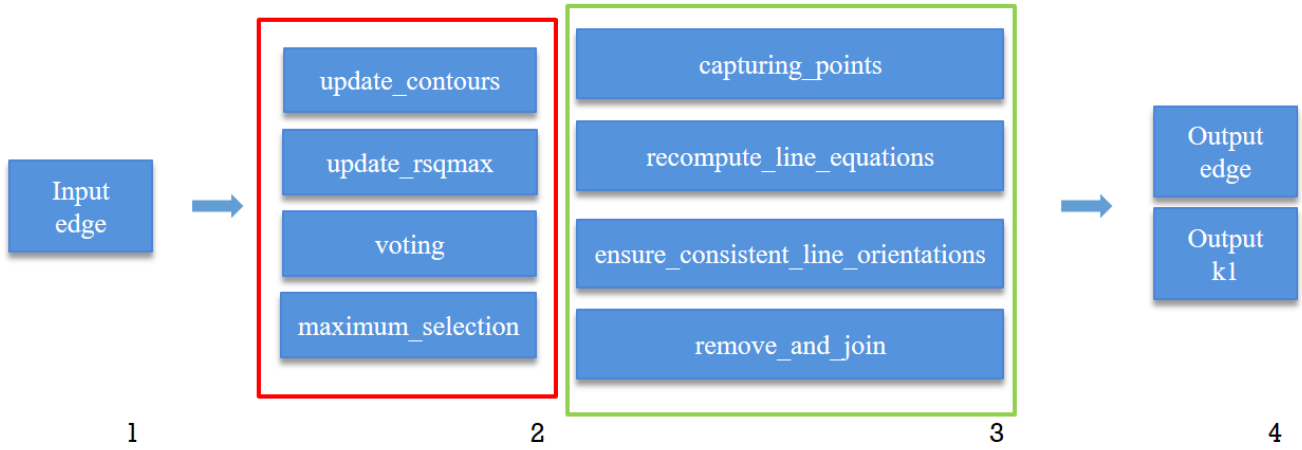

圖 6、Improved_hough 流程圖

1. 讀入輪廓
2. 進行畸變參數迭代計算
3. 進行輪廓整併
4. 輸出輪廓及參數

4.3 資料結構

功能描述

基本資料型態定義

資料結構

```
typedef char                I8;
typedef unsigned char       U8;
typedef short               I16;
typedef unsigned short      U16;
typedef int                 I32;
typedef unsigned int        U32;
typedef long long           I64;
typedef unsigned long long  U64;
typedef float               F32;
typedef double              F64;
```

4.3.1 Point2D

功能描述

存放兩個雙精度浮點數型的座標位置

資料結構

```
typedef struct {
	F64 x;
	F64 y;
}Point2D;
```

4.3.2 image_primitives

功能描述

存放多條直線組成的直線集、直線總條數、畸變模型

資料結構

```
typedef struct {
	line_Obj *lines;
	unsigned int capacity;
	unsigned int size;
	lens_distortion_model distortion;
}image_primitives;
```

4.3.3 line_Obj

功能描述

存放一個由多個像素點所組成的直線，及這條直線的直線方程式 ax+by+c=0 之係數 a、b、c

資料結構

```
typedef struct {
	double rect[3];
	line_Points *line_points;
}line_Obj;
```

4.3.4 line_Points

功能描述

可以存放多個 Point2D 組成的資料結構

資料結構

```
typedef struct {
	Point2D *points;
	unsigned int capacity;
	unsigned int size;
}line_Points;
```

4.3.5 lens_distortion_model

功能描述

存放三個畸變參數(1, k1, k2)，畸變中心點以及畸變模型種類(division、polynomial)

資料結構

```
typedef struct {
	double d[3];
	Point2D c;
	int type;
}lens_distortion_model;
```

4.3.6 image_XLD

功能描述

儲存輪廓點之資訊

資料結構

```
typedef struct
{
	unsigned int index_size;
	double* ptRows;
	double* ptCols;
	int *index;
	int width;
	int height;
	bool *c;
	float *cosine;
	float *sine;
}image_XLD;
```

4.4 Image_Primitives 相關函式

主要函式

4.4.1 Build_Image_Primitives

void Build_Image_Primitives(image_primitives *Image_Primitives, int line_size, int max_Points)

功能描述

創建並初始化一個 image_primitives 的資料結構

Input

- Image_Primitives (image_primitives *)：輸入已宣告之 Image_Primitives
- line_size (int)：輸入直線條數上限
- max_Points (int)：輸入一條直線內點數上限

Output

- Image_Primitives (image_primitives *)：輸出創建好的 Image_Primitives

設計方法

依照輸入的直線條數以及直線內點數，呼叫相應大小的記憶體空間，並回傳 Image_Primitives 的指標。

使用方法

輸入欲創建之 Image_Primitives，以及 Image_Primitives 內欲儲存的直線條數、直線內點數。

4.4.2 pass_line_Obj

void pass_line_Obj(line_Obj *dist_line, line_Obj *src_line, int numofLine)

功能描述

將一組 line_Obj 複製到另一組 line_Obj

Input

- dist_line (line_Obj *) : 輸入欲存放複製內容的 line_Obj
- src_line (line_Obj *) : 輸入欲複製的 line_Obj
- numofLine (int) : 輸入 src_line 的直線條數

Output

- dist_line (line_Obj *) : 輸出已複製來源內容的 line_Obj

設計方法

將 src_line 內所有參數一一傳入 dist_line。

使用方法

輸入欲複製及複製過去的變數，以及欲複製的直線條數。

4.4.3 pass_image_primitives

void pass_image_primitives(image_primitives *dist_image_primitives, image_primitives *src_image_primitives)

功能描述

將一組 image_primitives 複製到另一組 image_primitives

Input

- dist_image_primitives (image_primitives *) : 輸入欲存放複製內容的 image_primitives
- src_image_primitives (image_primitives *) : 輸入欲複製的 image_primitives

Output

- dist_image_primitives (image_primitives *)：輸出已複製來源內容的 image_primitives

設計方法

將 src_ image_primitives 內所有參數一一傳入 dist_ image_primitives。

使用方法

輸入欲複製及複製過去的變數。

4.5 指令功能說明

4.5.1 line_equation_distortion_extraction_improved_hough

```
double line_equation_distortion_extraction_improved_hough(
    image_XLD *canny_edges,
    image_primitives *image_primitive,
    float distance_point_line_max,
    int nlines,
    float angle_resolution,
    float distance_resolution,
    float initial_distortion_parameter,
    float final_distortion_parameter,
    float distortion_parameter_resolution,
    float angle_point_orientation_max_difference,
    lens_distortion_model ini_ldm)
```

功能描述

利用輸入的輪廓經過霍夫變換檢測直線來迭代出最佳的畸變參數 k1，用算出的畸變參數 k1 可得到畸變模型，再將輪廓透過畸變模型得到校正後的直線，並將這些直線進行整理合併的動作。

Input

- canny_edges (image_XLD *)：輸入 canny 計算出的輪廓
- image_primitive (image_primitives *) ：欲存放執行完此函數後得到的 image_primitive
- distance_point_line_max(float) ：判斷點到線距離之閥值(預設 3)
- nlines(int) ：檢測出的直線數量上限(預設 100)
- angle_resolution(float) ：角度離散化的 step(預設 0.1)
- distance_resolution(float) ：距離離散化的 step(預設 1)
- initial_distortion_parameter(float) ：畸變參數離散值的初始值

- final_distortion_parameter(float) ：畸變參數離散值的最終值
- distortion_parameter_resolution(float) ：畸變參數離散化的 step(預設 0.1)
- angle_point_orientation_max_difference(float) ：判斷角度差之閥值(預設 10)
- ini_ldm(lens_distortion_model) ：輸入之畸變模型

Output

- image_primitive (image_primitives *)：輸出計算完的畸變模型及直線群組

Return

double : 最高投票分數

設計方法

以下為 Radial Distortion 畸變模型公式，$k1$、$k2$、$k3$…為畸變參數，本計畫演算法計算參數數量為 2 個，亦即計算$k1$、$k2$，由於$k3$以後之參數數值非常小，影響影像畸變程度微小，因此本演算法將不計算剩下參數。

$$\begin{pmatrix}\hat{x}-x_c\\ \hat{y}-y_c\end{pmatrix}=L(r)\begin{pmatrix}x-x_c\\ y-y_c\end{pmatrix}$$

其中$\hat{x}$、$\hat{y}$為校正後的點座標，x、y為原始影像點座標，x_c、y_c為畸變中心點，$L(r)$為畸變模型。畸變模型$L(r)$分為 polynomial 以及 division 兩種模型，如以下公式：

$$polynomial:\ \ L(r)=1+k_1*r^2+k_2*r^4$$

$$division:\ \ L(r)=\frac{1}{1+k_1*r^2+k_2*r^4}$$

下圖為此函式主要流程圖，此函數使用霍夫變換演算法進行直線檢測，由於輸入的畸變影像輪廓有許多因失真被彎曲的線，使用傳統霍夫變換演算法會導致曲線被偵測成數段直線而得到錯誤結果，因此我們參考[1][2]所提出的 improved Hough transform 演算法，將畸變作為傳統二維霍夫變換的第三個維度，則空間中的直線由三個參數表示：到原點的距離、方向和畸變。

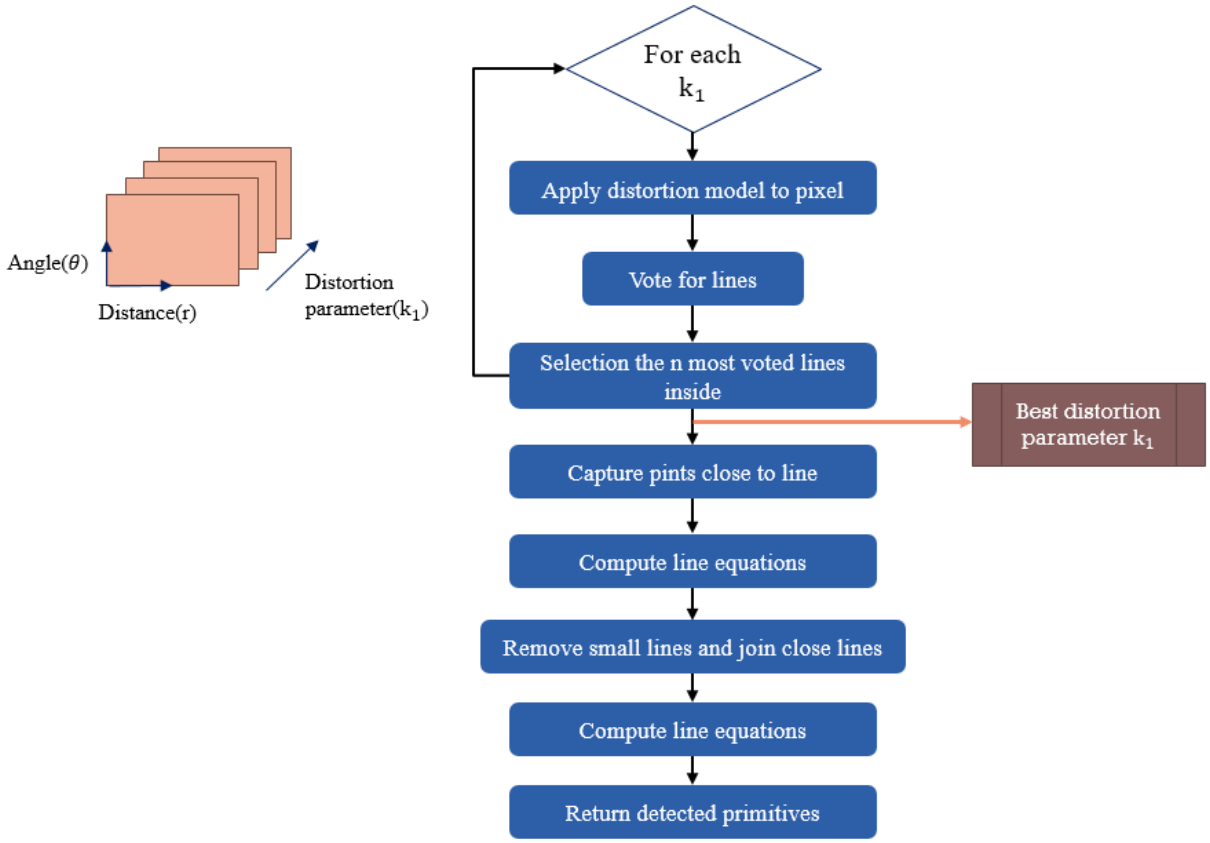

圖 7、Detection of lines with the improved Hough transform 流程架構圖

根據參數 initial distortion parameter、final distortion parameter 將 k1 的維度切成數個值($[k_1]_0$、$[k_1]_1$、$[k_1]_2$、$\cdots[k_1]_n$)，每次都以其中一個 k1 將輪廓作校正，再以校正後的輪廓點作霍夫變換中的投票，依次執行 n 個不同 k1 值。在迭代計算中畸變參數使用 One-parameter model，估算出第一個畸變參數 k1，並針對投票出來的直線進行整併。

使用方法

輸入 canny edge、初始畸變模型，則可得到計算出的最佳畸變參數 k1，以及經過此畸變模型校正、整理合併後的直線。

4.5.2 update_contours

void update_contours(lens_distortion_model ini_ldm, image_XLD *canny_edges, image_XLD *contours_modified, int *index, int index_size)

功能描述

透過輸入的畸變模型 ini_ldm，校正原本的 canny edge，並存入 contours_modified 中

Input

- ini_ldm (lens_distortion_model)：輸入畸變模型

- canny_edges (image_XLD *)：輸入原始輪廓
- index(int *)：輸入原始 canny edge 中輪廓點的 index
- index_size(int)：輸入 index 一維陣列的長度

Output

- contours_modified (image_XLD *)：儲存校正後的輪廓

Return

無

設計方法

若輸入畸變模型沒有值，則將 canny_edges 複製到 contours_modified 中；若有畸變參數，則計算 canny_edges 中每一個點經過校正後的點座標及 orientation，存入 contours_modified 中，其中 orientation 只表示 0~180 度，因此 cosine 與 sine 在大於 180 度時要乘上一個負號。

使用方法

輸入原始輪廓及畸變模型，則會得到經畸變模型校正後的輪廓。

4.5.3 evaluation

Point2D evaluation(Point2D point, lens_distortion_model model)

功能描述

將輸入的點座標經過輸入的畸變模型得到校正後的點座標

Input

- point (Point2D)：點座標
- model(lens_distortion_model)：畸變模型

Output

無

Return

Point2D：校正後的點座標

設計方法

透過輸入的點座標及畸變模型中心點，算出畸變模型公式中的r^2，再通過函式 polynomial_evaluation 算出 polynomial model($1 + k_1 * r^2 + k_2 * r^4$)的值，而 division model 就是 polynomial model 的倒數，最後再用算出的 model 透過以下公式得到校正後的點座標。

$$\begin{pmatrix}\hat{x}-x_c\\ \hat{y}-y_c\end{pmatrix}=L(r)\begin{pmatrix}x-x_c\\ y-y_c\end{pmatrix}$$

$$polynomial:\ \ L(r)=1+k_1*r^2+k_2*r^4$$

$$division:\ \ L(r)=\frac{1}{1+k_1*r^2+k_2*r^4}$$

使用方法

輸入點座標及畸變模型，則可得到校正後的點座標。

4.5.4 polynomial_evaluation

double polynomial_evaluation(double *a, int Na, double x)

功能描述

計算 polynomial model$\left(1+k_1*r^2+k_2*r^4\right)$

Input

- a (double *) : 畸變參數(k1, k2)
- Na(int) : 總共有幾個畸變參數
- x(double) : r^2

Output

無

Return

double : 計算出 polynomial model 的值

設計方法

透過輸入參數的個數，跑迴圈算出$(1+k_1*r^2+k_2*r^4)$的值。

使用方法

輸入畸變參數、畸變參數個數以及 r 平方，輸出可得 polynomial model 公式的值。

4.5.5 orientation_update

Point2D orientation_update(Point2D point, float sine, float cosine, lens_distortion_model model)

功能描述

透過輸入的畸變模型，校正輸入的輪廓點的 orientation

Input

- point (Point2D) : 輸入點座標

- sine(float) : 輸入點 y 方向的 orientation
- cosine(float) : 輸入點 x 方向的 orientation
- model(lens_distortion_model) : 畸變模型

Output

無

Return

Point2D : 校正後點的 orientation

設計方法

先將輸入點座標透過畸變模型得到校正後點座標 p_prime，再將輸入點座標加上此點的 orientation，透過校正模型得到校正後 orientation 點座標 p_ori_prime，p_ori_prime 與 p_prime 做相減，得到 x、y 方向上的向量 a、b，則校正後點的 orientation 為$\left(\frac{a}{\sqrt{a^2+b^2}},\frac{b}{\sqrt{a^2+b^2}}\right)$。

使用方法

輸入點座標、畸變模型、此點座標的 orientation，則可得校正後點座標之 orientation。

4.5.6 update_rsqmax

double update_rsqmax(Point2D center, int w, int h)

功能描述

計算輸入中心點到影像四個頂點的距離平方，回傳最大的距離平方值

Input

- center (Point2D) : 畸變中心點
- w(int) : 影像的 width
- h(int) : 影像的 height

Output

無

Return

double : 中心點與距中心點最遠角的距離平方

設計方法

將輸入的中心點分別與影像四個頂點做距離平方，並回傳最大值。

使用方法

輸入中心點及影像長寬，輸出中心點與距中心點最遠角的距離平方。

4.5.7 norm

double norm(Point2D k)

功能描述

計算輸入點座標與原點的歐式距離

Input

- k(Point2D) : 輸入點座標

Output

無

Return

double : 回傳輸入點座標與原點之歐式距離值

設計方法

以下方公式計算輸入點座標與原點之歐式距離，並回傳其結果。

$$\|(x,y)\| = \sqrt{x^2+y^2}$$

使用方法

輸入一點，輸出此點與原點之歐式距離。

4.5.8 norm2

float norm2(Point2D k)

功能描述

計算輸入點座標與原點的歐式距離平方

Input

- k(Point2D) : 輸入點座標

Output

無

Return

float : 回傳輸入點座標與原點之歐式距離值平方

設計方法

以下方公式計算輸入點座標與原點之歐式距離平方，並回傳其結果。

$$\|(x,y)\|^2 \quad = \quad x^2 + y^2$$

使用方法

輸入一點，輸出此點與原點之歐式距離值平方。

4.5.9 substract

Point2D substract(Point2D a, Point2D b)

功能描述

將兩個輸入點座標作相減

Input

- a(Point2D)：輸入點座標
- b(Point2D)：輸入點座標

Output

無

Return

Point2D：回傳 a、b 兩點相減後的點座標

設計方法

輸入兩點 $\boldsymbol{a}\,(a_x, a_y)$、$\boldsymbol{b}\,(b_x, b_y)$，做下列相減運算：

$$\boldsymbol{c} = \boldsymbol{a} - \boldsymbol{b} = (a_x - b_x, a_y - b_y)。$$

回傳相減後之 $\boldsymbol{c}$ 點座標。

使用方法

輸入兩個點座標，輸出相減後的點座標。

4.5.10 voting

void voting(image_XLD *contours_modified, lens_distortion_model it_ldm,
double step_angle, int height_score, int width_score, float angle_increment,
float distance_resolution, double *sine, double *cosine, float **score_k, int *index, int index_size)

功能描述

針對校正後的輪廓進行霍夫變換的投票。每個點都在給定的角度範圍內投票，而不是 360 度都投，且給定此點周圍一個距離內皆可投票，投票分數的權重與距離成反比。

Input

- contours_modified (image_XLD *) : 輸入輪廓
- it_ldm (lens_distortion_model) : 輸入畸變模型
- step_angle(double) : 角度的步伐(徑度)
- height_score(int) : 儲存各個角度的投票分數
- width_score(int) : 儲存各個距離的投票分數
- angle_increment(float) : 欲投票的角度範圍
- distance_resolution(float) : 距離解析度
- sine(double *) : 0~180 度的 sine 值查表
- cosine(double *) : 0~180 度的 cosine 值查表
- index(int *) : 輸入原始 canny edge 中輪廓點的 index
- index_size(int) : index 一維陣列的長度

Output

score_k(float **) : 存放各個角度與距離投票值的二維矩陣

Return

無

設計方法

透過輸入的畸變模型對輸入的輪廓進行點座標及 orientation 校正，再以校正後的點依序進行投票。投票有限制角度及距離範圍，若角度太大則不予以投票，角度限制在正負 10 度之間。而距離投票原本是要通過此點才能投票，此函式多給了一個範圍讓周圍的距離也可投，但分數就不是一票，而是與距離成反比，意即距離越遠，投票分數越低。

使用方法

輸入輪廓及畸變模型，和判斷用的閥值，則輸出數個角度及距離的投票分數矩陣。

4.5.11 maximum_selection

void maximum_selection(float **score_k, int *i_pos, int *j_pos, int pind, double *max_score, double *votation_score, int width_score, int height_score, int nlines_plus, float distance_resolution, float distance_point_line_max, float angle_point_orientation_max_difference, float angle_resolution, double *sine, double *cosine, line_Obj *lines)

功能描述

透過投票結果的矩陣 score_k，選擇出投票分數最高的前 nlines_plus (預設 100)條線

Input

- score_k (float **) : 存放不同角度不同距離的投票分數
- i_pos(int *) : 投票分數大於 10 的角度
- j_pos(int *) : 投票分數大於 10 的距離
- pind(int) : 投票分數大於 10 的數量
- max_score(double *) : 欲儲存最大投票分數
- width_score(int) : 存放各個距離的陣列大小
- height_score(int) : 存放各個角度的陣列大小
- nlines_plus(int) : 欲計算的直線數量
- distance_resolution(float) : 距離離散化的 step(預設 1)
- distance_point_line_max(float) : 判斷點到線距離之閥值(預設 3)
- angle_point_orientation_max_difference(float) : 判斷角度差之閥值(預設 10)
- angle_resolution(float) : 度離散化的 step(預設 0.1)
- sine(double *s) : 0~180 度的 sine 值查表
- cosine(double *s) : 0~180 度的 cosine 值查表

Output

- votation_score(double *) : 在當下的畸變參數 k1 之下算出的 nlines_plus 條線投票總數
- lines(line_Obj *) : 存入前 nlines_plus 條線的直線方程式係數

Return

無

設計方法

由函式 voting 得到當下 k1 的數個 d、θ(距離、角度)投票結果，選取出投票數最多的前 nlines_plus 條線(預設為 100 條線)，此 nlines_plus 條線根據 d、θ 可得線方程式 ax+by+c=0 之係數 a、b、c，並將此 nlines_plus 條線所擁有投票數累加，設為此 k1 的 votation score，比較每個 k1 得出之最高的 votation score，此時的 k1 即為最佳的 k1 值。

使用方法

輸入投票結果 score_k、投票分數大於 10 之 d、θ index 矩陣，則輸出前 nlines_plus 條最高分數之直線，以及投票總分 votation score。

4.5.12 capturing_points

void capturing_points(image_XLD *contours_modified, image_XLD *contours, float angle_point_orientation_max_difference, image_primitives *image_primitive,
image_primitives *image_primitive_corrected, image_primitives *image_primitive_original, int nlines_plus, float max_distance, int *index, int index_size)

功能描述

將輪廓點依據直線方程式分群，利用點的方向性、點到線距離等資訊來決定此點屬於哪一條直線。

Input

- contours_modified (image_XLD *) : 校正後輪廓
- contours(image_XLD *) : 原始 canny 輪廓
- angle_point_orientation_max_difference(float) : 判斷角度差之閥值(預設 10)
- image_primitive(image_primitives *) : 提供畸變模型參數
- nlines_plus(int) : 直線條數
- max_distance(float) : 判斷點到直線距離的閥值
- index(int *) : 輸入原始 canny edge 中輪廓點的 index
- index_size(int) : index 一維陣列的長度

Output

- image_primitive(image_primitives *) : 存放分群後的 contours_modified 的點
- image_primitive_corrected(image_primitives *) : 存放分群後經過校正的點
- image_primitive_original(image_primitives *) : 存放分群後原始 canny 輪廓的點

Return

無

設計方法

此函式要將所有輪廓點歸類到這 N 條線中，若有輪廓點沒有被歸到任何一條線中，即捨棄此輪廓點。歸類依據兩者條件判斷，其一為此輪廓點之 orientation 與線之法向量作內積，內積值要大於閥值，其二為此輪廓點到此線距離要小於閥值，兩者條件皆符合便將此點歸為此線之點，以下圖為例，只有 p1 滿足條件，則 p1 會被歸到 line 裡，p2、p3 則會繼續看別條線是否滿足。

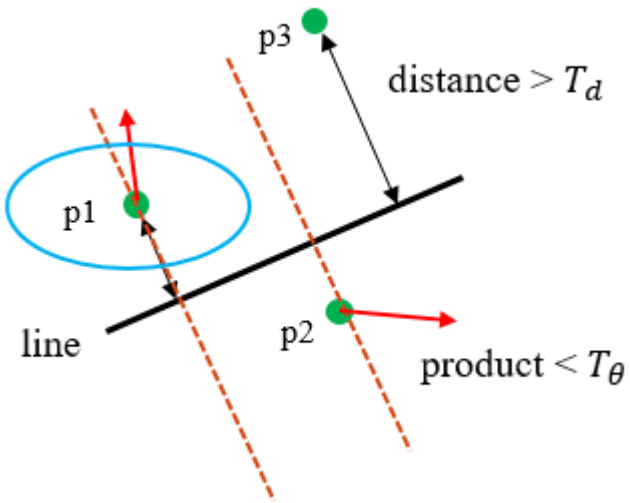

圖 8、點歸類判斷示意圖

使用方法

輸入要儲存輪廓點的三個 image_primitives、原本輪廓、校正後輪廓以及判斷用參數，則可得三個 image_primitives 輸出，其中已儲存每條線中的輪廓點。

4.5.13 Point2Line_distance

double Point2Line_distance(Point2D point, line_Obj line)

功能描述

計算點到直線距離

Input

- point (Point2D)：點座標
- line(line_Obj)：直線

Output

無

Return

double：回傳點到直線距離

設計方法

直線 line_Obj 中有儲存此線之直線方程式係數，利用點到直線距離公式算出輸入點到輸入線之距離。

使用方法

輸入點及直線，則可得點到直線距離。

4.5.14 line_point_push_back

void line_point_push_back(line_Obj *line, Point2D add_point)

功能描述

將一個點加到一條線裡

Input

- line (line_Obj *) : 欲加入點的直線
- add_point(Point2D) : 欲加入的點座標

Output

- line (line_Obj *) : 已將 add_point 加入的直線

Return

無

設計方法

將欲加入的點存到輸入線中，接續在此線原本點之後，且將此線點數加一。

使用方法

輸入一點及一線，輸出為將此點加到此線裡。

4.5.15 ensure_consistent_line_orientations

void ensure_consistent_line_orientations(image_primitives *image_primitive, image_primitives *image_primitive_corrected, image_primitives *image_primitive_original, image_XLD *contours, int width)

功能描述

確定一條直線上的每個點方向有無一致，若有方向不一致的點，將予以刪除

Input

- image_primitive (image_primitives *) : 欲判斷線上點方向並刪除的 image_primitives
- image_primitive_corrected (image_primitives *) : 欲判斷線上點方向並刪除的 image_primitives_corrected
- image_primitive_ original (image_primitives *) : 欲判斷線上點方向並刪除的 image_primitives_ original
- contours (image_XLD *) : 原始 canny edge

- width(int)：影像的寬度

Output

- image_primitive (image_primitives *)：經過刪除點動作的 image_primitives
- image_primitive_corrected (image_primitives *)：經過刪除點動作的 image_primitives_corrected
- image_primitive_ original (image_primitives *)：經過刪除點動作的 image_primitives_ original

Return

無

設計方法

針對輸入每一條線上的點進行方向確認，確保被歸類成同一條線的所有點方向保持一致性。先對線上每一點的 orientation 與此線的方向向量作內積，根據內積結果的正負值當作此點的 orientation sign，計數線上所有點的 orientation sign 是正數多還是負數多，保留多數的那個方向，少數方向的點則被刪除，以下圖為例，內積結果為負的點為多數，因此會將最右邊兩個點刪去，此線便剩下左邊五個點。

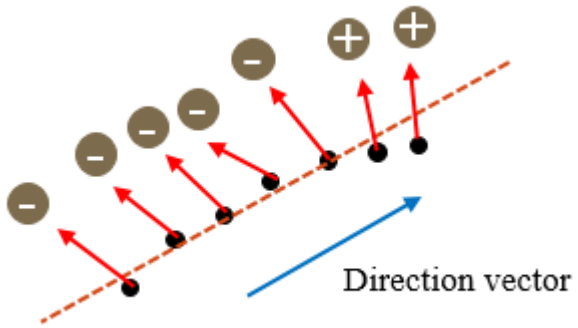

圖 9、線上點方向確認示意圖

使用方法

輸入欲判斷線上點方向的 image_primitives，經過計算後會輸出已將方向不一致之點刪去的 image_primitives。

4.5.16 compute_orientation_sign

double compute_orientation_sign(line_Obj *line, int width, int ip, image_XLD *contours)

功能描述

對線上一點的 orientation 與此線的方向向量作內積

Input

- line (line_Obj *) : 欲作內積之直線
- width(int) : 影像的寬
- ip(int) : 欲作內積之線上一點
- contours(image_XLD *) : 原始 canny edge

Output

無

Return

double : 回傳內積後的結果

設計方法

先取得輸入線上一點的 orientation，以及透過輸入線的直線方程式係數取得方向向量，兩者作向量內積。

使用方法

輸入線條、欲計算方向之線上點的 index，則回傳點的 orientation 與線的方向向量內積結果。

4.5.17 erase_point

void erase_point(image_primitives *image_primitive_original, int line_index, int *point_index)

功能描述

將一個點從一條線中刪去

Input

- image_primitive_original (image_primitives *) : 欲刪除當中某條線上某一點的 image_primitive_original
- line_index (int) : 欲在 image_primitive_original 中哪條線刪除點
- point_index (int *) : 欲刪除點在線上的 index

Output

- image_primitive_original (image_primitives *) : 經過點刪除的 image_primitive_original
- ip(int *) : 回傳刪除點 index-1

Return

無

設計方法

假設要刪去第 ip 個點，則將 ip+1 到最後一個點依序往前移一個，移完後將最後一個點歸零，並將點數量減一，且將 ip-1 回傳，為使此函數上一層的迴圈 index+1 後能判斷第 ip 個點。

使用方法

輸入欲刪除點的 image_primitives，已及此點位於第幾條線中的第幾個點，則輸出為已刪去此點的 image_primitives。

4.5.18 recompute_line_equations

void recompute_line_equations(image_primitives *image_primitive, image_primitives *image_primitive_corrected, image_primitives *image_primitive_original)

功能描述

將 image_primitive 中每一條線重新計算其直線方程式

Input

- image_primitive (image_primitives *) : 欲重新計算直線方程式之 image_primitive
- image_primitive_corrected (image_primitives *) : 欲重新計算直線方程式之 image_primitives_corrected
- image_primitive _original (image_primitives *) : 欲重新計算直線方程式之 image_primitives_original

Output

- image_primitive (image_primitives *) : 已重新計算完直線方程式之 image_primitive
- image_primitive_corrected (image_primitives *) : 已重新計算完直線方程式之 image_primitive_corrected
- image_primitive _original (image_primitives *) : 已重新計算完直線方程式之 image_primitive_original

Return

無

設計方法

此函式依序將每一條線傳入函式 points_to_equation，計算出此線的直線方程式，並把直線方程式的係數存入三個 image_primitives 中。

使用方法

輸入三個 image_primitives，則會輸出 image_primitive_corrected 中每條線的直線方程式，並存入三個 image_primitives 中。

4.5.19 points_to_equation

double points_to_equation(line_Obj *line)

功能描述

計算輸入直線的直線方程式

Input

- line (line_Obj *)：輸入直線

Output

- line (line_Obj *)：輸出已重新計算直線方程式之直線

Return

回傳平均點到直線距離

設計方法

本演算法使用最小平方法-垂直偏移(Least Squares Fitting--Perpendicular Offsets)求出直線方程式$y = a + bx$的係數a與b，示意圖及推導如下：

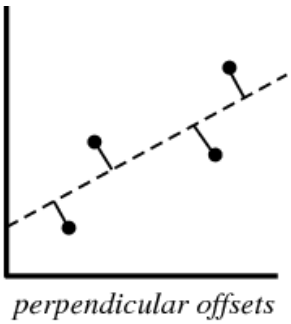

圖 10、Perpendicular Offsets 示意圖

點到線距離平方和	$R_{\perp}^{2} = \sum_{i=1}^{n} \frac{[y_i - (a + bx_i)]^2}{1 + b^2}$

找 minimum，對 a 偏微=0 時	$\frac{\partial R_{\perp}^2}{\partial a}=\frac{2}{1+b^2}\sum_{i=1}^{n}[y_i-(a+bx_i)]\,(-1)=0$
找 minimum，對 b 偏微=0 時	$\frac{\partial R_{\perp}^2}{\partial b}=\frac{2}{1+b^2}\sum_{i=1}^{n}[y_i-(a+bx_i)]\,(-x_i)+\sum_{i=1}^{n}\frac{[y_i-(a+bx_i)]^2(-1)(2b)}{(1+b^2)^2}=0$
化簡上式	$a=\frac{\sum_{i=1}^{n}y_i-b\sum_{i=1}^{n}x_i}{n}=\bar{y}-b\bar{x}$
化簡上式	$(1+b^2)\sum_{i=1}^{n}[y_i-(a+bx_i)]x_i+b\sum_{i=1}^{n}[y_i-(a+bx_i)]^2=0$
代換化簡上式	$b^2+\frac{\sum_{i=1}^{n}y_i^2-\sum_{i=1}^{n}x_i^2+\frac{1}{n}[(\sum_{i=1}^{n}x_i)^2-(\sum_{i=1}^{n}y_i)^2]}{\frac{1}{n}\sum_{i=1}^{n}x_i\sum_{i=1}^{n}y_i-\sum_{i=1}^{n}x_iy_i}b-1=0$
解 b	$b=-B\pm\sqrt{B^2+1}$ $B=\frac{1}{2}\frac{(\sum_{i=1}^{n}y_i^2-n\bar{y}^2)-(\sum_{i=1}^{n}x_i^2-n\bar{x}^2)}{n\bar{x}\bar{y}-\sum_{i=1}^{n}x_iy_i}$

使用方法

輸入欲計算直線方程式之直線，計算出的直線方程式係數會直接存到輸入的 line_Obj 中，並回傳每一點到計算出的直線方程式之平均距離

4.5.20 remove_and_join

void remove_and_join(image_primitives *image_primitive, image_primitives *image_primitive_corrected, image_primitives *image_primitive_original,
float angle_point_orientation_max_difference, float distance_point_line_max, int width, image_XLD *contours)

功能描述

將輸入的 image_primitive 中所有直線作刪去或合併的動作

Input

- image_primitive (image_primitives *)：輸入欲作線條整併之 image_primitive
- image_primitive_corrected (image_primitives *)：輸入欲作線條整併之 image_primitive

- image_primitive_original (image_primitives *) : 輸入欲作線條整併之 image_primitive
- angle_point_orientation_max_difference(float) : 判斷角度差之閥值(預設 10)
- distance_point_line_max(float) : 判斷點到線距離之閥值(預設 3)
- width(int) : 影像的寬
- contours(image_XLD *) : 原始 canny edge

Output

- image_primitive (image_primitives *) : 輸出已作線條整併之 image_primitive
- image_primitive_corrected (image_primitives *) : 輸出已作線條整併之 image_primitive
- image_primitive_original (image_primitives *) : 輸出已作線條整併之 image_primitive

Return

無

設計方法

於 image_primitive 中所有線條依次判斷此線點數是否夠多，不夠則刪除，夠的話便以此線為基準，此線之後的每一條線都分別跟此線判斷是否可以合併成同條線。判斷是合併有三個條件如下：利用兩條線法向量作內積取得之間的夾角關係，夾角要小於閥值；其中一線的所有點到另一線距離要小於閥值；其中一線所有點與另一線所有點距離總和要大於閥值，以防止連到兩條平行的線。假設當下為第 k 條線，則第 k+1 到第 N 條線都依次與第 k 條線作比較判斷，符合三個條件者便將所有點加到第 k 條線中，再刪除此線。

使用方法

輸入要做線條整併的 image_primitive，以及判斷是否要刪去或合併的閥值，則輸出 image_primitive 中儲存的線便是經過整併後的結果。

4.5.21 erase_line

void erase_line(line_Obj *lineObj, int line_size, int ip)

功能描述

將一個包含多條線條的線條群組中，刪去其中一條線

Input

- lineObj (line_Obj *) : 欲刪除其中一條線的線條群組
- line_size(int) : 輸入線條群組的總線條數
- ip(int) : 欲刪除線在線條群組中的 index

Output

- lineObj (line_Obj *) : 已刪除線條後的線條群組

Return

無

設計方法

假設要刪去第 ip 條線，則將 ip+1 到最後一條線依序往前移一條，移完後將最後一條歸零，並將線條數量減一。

使用方法

輸入線條群組，以及欲刪去的線在線條群組中的 index，則輸出的線條群組已刪去此線且後面的線皆往前移。

4.5.22 Point2Point_distance_min

double Point2Point_distance_min(line_Obj line, Point2D point)

功能描述

計算一點到一線上所有點的距離，取最小值

Input

- line (line_Obj) : 線條
- point(Point2D) : 點座標

Output

無

Return

double : 回傳一點到一線上所有點的距離的最小值

設計方法

分別計算輸入點與輸入線上所有點的距離，儲存最小值並回傳。

使用方法

輸入一線條與一點，則輸出為一點到一線上所有點的距離的最小值。

4.5.23 combine_line

void combine_line(line_Obj *lineObj, line_Obj *lineObj_add)

功能描述

將輸入的線條 lineObj_add 加入到輸入線條 lineObj

Input

- lineObj (line_Obj *)：被加入線條
- lineObj_add(line_Obj *)：要加入的線條

Output

- lineObj (line_Obj *)：輸出已加入另一條線的 lineObj

Return

無

設計方法

將 lineObj_add 所有點依序存入 lineObj 裡，接續在 lineObj 原本的點後面，且 lineObj 的點數量更新為原本點數量加 lineObj_add 點數量。

使用方法

輸入被加入線條與要加入線條，則輸出的被加入線條中已包含要加入線條所有點。

5 Parameter_optimization

5.1 軟體架構

如下圖所示，本函式主要迭代更新k_1、k_2、c_x、c_y四個參數，每更新一組新的(k_1, k_2, c_x, c_y)，便會重新利用第二部份後半演算法(Capturing points 到 Recompute line equations)更新校正後輪廓，並將所有線條內的所有點個數作累加，當作判斷更新的參數是否有變好的一個依據。

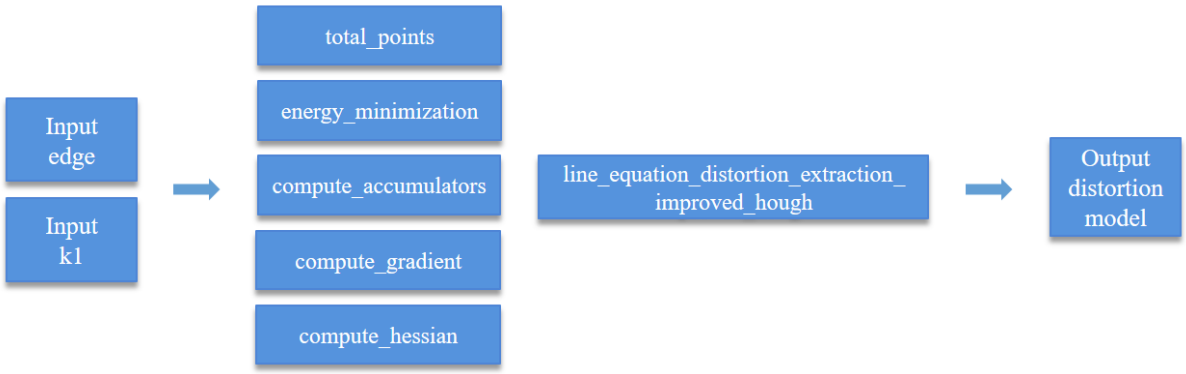

圖 11、Parameter_optimization 架構圖

5.2 軟體流程

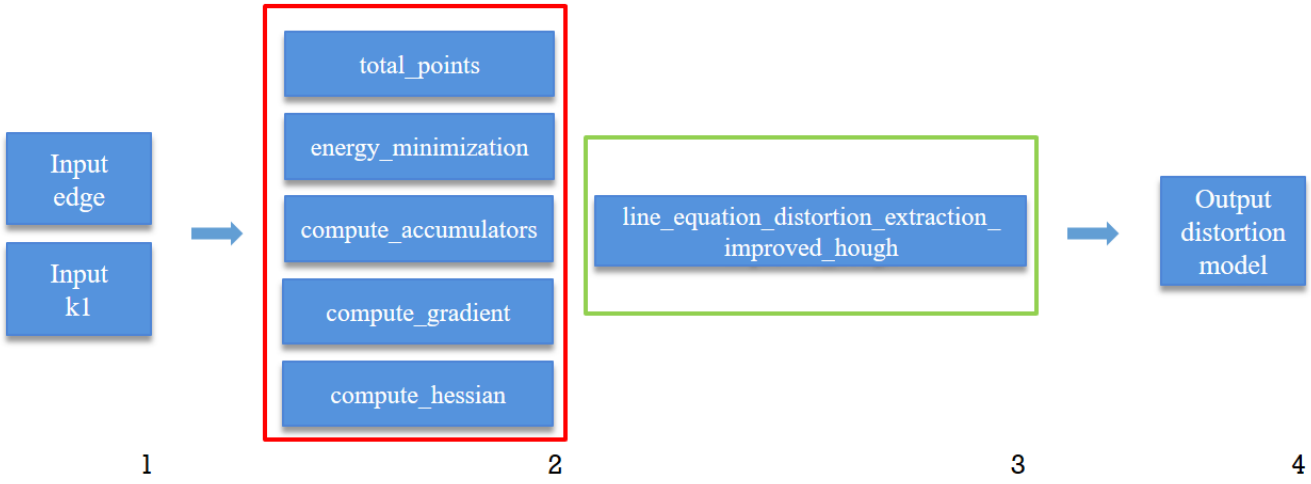

圖 12、Parameter_optimization 流程圖

1. 讀入上一步驟算出之輪廓及畸變參數 k1
2. 迭代更新 k1、k2、cx、cy 四個參數
3. 利用上一步驟輪廓整併之演算法取得輪廓以判斷迭代是否繼續
4. 輸出最佳畸變模型參數

5.3 資料結構

功能描述

基本資料型態定義

資料結構

```
typedef char                I8;
typedef unsigned char       U8;
typedef short               I16;
typedef unsigned short      U16;
typedef int                 I32;
typedef unsigned int        U32;
typedef long long           I64;
typedef unsigned long long  U64;
typedef float               F32;
typedef double              F64;
```

5.3.1 Point2D

功能描述

存放兩個雙精度浮點數型的座標位置

資料結構

```
typedef struct {
	F64 x;
	F64 y;
}Point2D;
```

5.3.2 image_primitives

功能描述

存放多條直線組成的直線集、直線總條數、畸變模型

資料結構

```
typedef struct {
	line_Obj *lines;
	unsigned int capacity;
	unsigned int size;
	lens_distortion_model distortion;
}image_primitives;
```

5.3.3 line_Obj

功能描述

存放一個由多個像素點所組成的直線，及這條直線的直線方程式 ax+by+c=0 之係數 a、b、c

資料結構

```
typedef struct {
	double rect[3];
	line_Points *line_points;
}line_Obj;
```

5.3.4 line_Points

功能描述

可以存放多個 Point2D 組成的資料結構

資料結構

```
typedef struct {
	Point2D *points;
	unsigned int capacity;
	unsigned int size;
}line_Points;
```

5.3.5 lens_distortion_model

功能描述

存放三個畸變參數(1、k1、k2)，畸變中心點以及畸變模型種類(division、polynomial)

資料結構

```
typedef struct {
	double d[3];
	Point2D c;
	int type;
}lens_distortion_model;
```

5.3.6 image_XLD

功能描述

儲存輪廓點之資訊

資料結構

```
typedef struct
{
	unsigned int index_size;
	double* ptRows;
	double* ptCols;
	int *index;
	int width;
	int height;
	bool *c;
	float *cosine;
	float *sine;
}image_XLD;
```

5.3.7 E_accumulators

功能描述

存放計算 energy 會使用到的參數

資料結構

```
typedef struct {
	double E_p1_p2_xc_yc, E_p1plush_p2_xc_yc, E_p1_p2plush_xc_yc,
		E_p1_p2_xcplush_yc, E_p1_p2_xc_ycplush, E_p1minush_p2_xc_yc,
		E_p1_p2minush_xc_yc, E_p1_p2_xcminush_yc, E_p1_p2_xc_ycminush,
		E_p1plush_p2plush_xc_yc, E_p1plush_p2_xcplush_yc,
		E_p1plush_p2_xc_ycplush, E_p1_p2plush_xcplush_yc,
		E_p1_p2plush_xc_ycplush, E_p1_p2_xcplush_ycplush, E1_p1_p2_xc_yc;
}E_accumulators;
```

5.4 Image_Primitives 相關函式

主要函式

5.4.1 Build_Image_Primitives

void Build_Image_Primitives(image_primitives *Image_Primitives, int line_size, int max_Points)

功能描述

創建並初始化一個 image_primitives 的資料結構

Input

- Image_Primitives (image_primitives *)：輸入已宣告之 Image_Primitives
- line_size (int)：輸入直線條數上限
- max_Points (int)：輸入一條直線內點數上限

Output

- Image_Primitives (image_primitives *)：輸出創建好的 Image_Primitives

設計方法

依照輸入的直線條數以及直線內點數，呼叫相應大小的記憶體空間，並回傳 Image_Primitives 的指標。

使用方法

輸入欲創建之 Image_Primitives，以及 Image_Primitives 內欲儲存的直線條數、直線內點數。

5.4.2 pass_line_Obj

void pass_line_Obj(line_Obj *dist_line, line_Obj *src_line, int numofLine)

功能描述

將一組 line_Obj 複製到另一組 line_Obj

Input

- dist_line (line_Obj *) : 輸入欲存放複製內容的 line_Obj
- src_line (line_Obj *) : 輸入欲複製的 line_Obj
- numofLine (int) : 輸入 src_line 的直線條數

Output

- dist_line (line_Obj *) : 輸出已複製來源內容的 line_Obj

設計方法

將 src_line 內所有參數一一傳入 dist_line。

使用方法

輸入欲複製及複製過去的變數，以及欲複製的直線條數。

5.4.3 pass_image_primitives

void pass_image_primitives(image_primitives *dist_image_primitives, image_primitives *src_image_primitives)

功能描述

將一組 image_primitives 複製到另一組 image_primitives

Input

- dist_image_primitives (image_primitives *) : 輸入欲存放複製內容的 image_primitives
- src_image_primitives (image_primitives *) : 輸入欲複製的 image_primitives

Output

- dist_image_primitives (image_primitives *) : 輸出已複製來源內容的 image_primitives

設計方法

將 src_ image_primitives 內所有參數一一傳入 dist_ image_primitives。

使用方法

輸入欲複製及複製過去的變數。

5.5 指令功能說明

5.5.1 iterative_optimization

double iterative_optimization(image_XLD *canny_edges, image_primitives *i_primitives, float distance_point_line_max_hough, int max_lines, float angle_resolution, float distance_resolution, float distortion_parameter_resolution, float angle_point_orientation_max_difference)

功能描述

藉由多次迭代運算逼近得到更好的校正參數值(k_1, k_2, C_x, C_y)。

Input

- canny_edges (image_XLD *) : 校正前的 edge points

- i_primitives (image_primitives *) : 校正線條與對應 edge points 資訊，並包含了校正模型(以第二階段所得到的模型為最佳化迭代的初始狀態)
- distance_point_line_max_hough (float) : 同第二階段設定
- max_lines (int) : 同第二階段設定
- angle_resolution (float) : 同第二階段設定
- distance_resolution (float) : 同第二階段設定
- distortion_parameter_resolution (float) : 同第二階段設定
- angle_point_orientation_max_difference (float) : 同第二階段設定

Output

- i_primitives (image_primitives *) : 藉由多次最佳化迭代後所更新的校正模型。

Return

- avg_error (double) : 第三部分的 error 指的是 edge points 離所歸屬線條的距離的總和，代表 edge points 對於線條的擬合程度(值越小表示擬合越小)。而線條越直則對於 edge points 的距離越小，代表校正效果越好。因此，在校正參數的最佳化的過程中，使用 edge points 到對應線條的距離總合做為 loss function 的衡量。

設計方法

在最佳化參數的過程上，利用兩項指標作為校正結果的衡量: (1) error: edge points 到其對應線條的距離總合。若校正效果越好，則 edge points 與線條擬合越好，所計算得到對應線條的距離(error)越小。最佳化的過程中會以計算得到的 error 作為最佳化的 loss function 已逼近更好的校正參數。 (2) num_points: 校正結果越好，則校正後的線條越趨近一正直線。在計算 edge points 到對應直線的距離越小，就能收集到更多有歸屬線條的 edge points。因此，程式中的 num_points 增加越多代表校正結果改善越多。最佳化的迭代過程中利用每次迭代後的模型所能新收集到的 edge points 數目(num_points/next_num_points)是否有所增長決定是否要繼續迭代或收斂。

首先，第三步在做校正參數最佳化時，會先使用第二階段所得到最好的校正 k_1 值作為最佳化的初始值。最佳化收斂的條件為: 連續 3 次更新的結果皆未多出前次收集點之數量比例的 1%。函數 energy_minimization 會藉由當前模型重新計算 edge points 與對應的線條，以 edge points 到對應線條的距離總合(error)做為最佳化指標 loss functionu 以逼近最佳校正參數(k_1, k_2, C_x, C_y)。

特別的是，第三階段中會呼叫第二階段的 line_equation_distortion_extraction_improved_hough 函數並利用特殊參數設定(initial_distortion_parameter, final_distortion_parameter 皆設定為 0)略過 3D Hough Transform 的階段，主要針對最佳化更新後的校正模型重新計算 edge points 與對應線條並進行線條的清除(clean)與整併(join)。

最後，最佳化過程中會以更新的校正模型是否能收集到更多的 edge points 作為校正好壞的判斷條件決定最佳化是否停止，並以收集到最多 edge points 的模型作為最佳校正參數輸出至第四階段作影像校正。

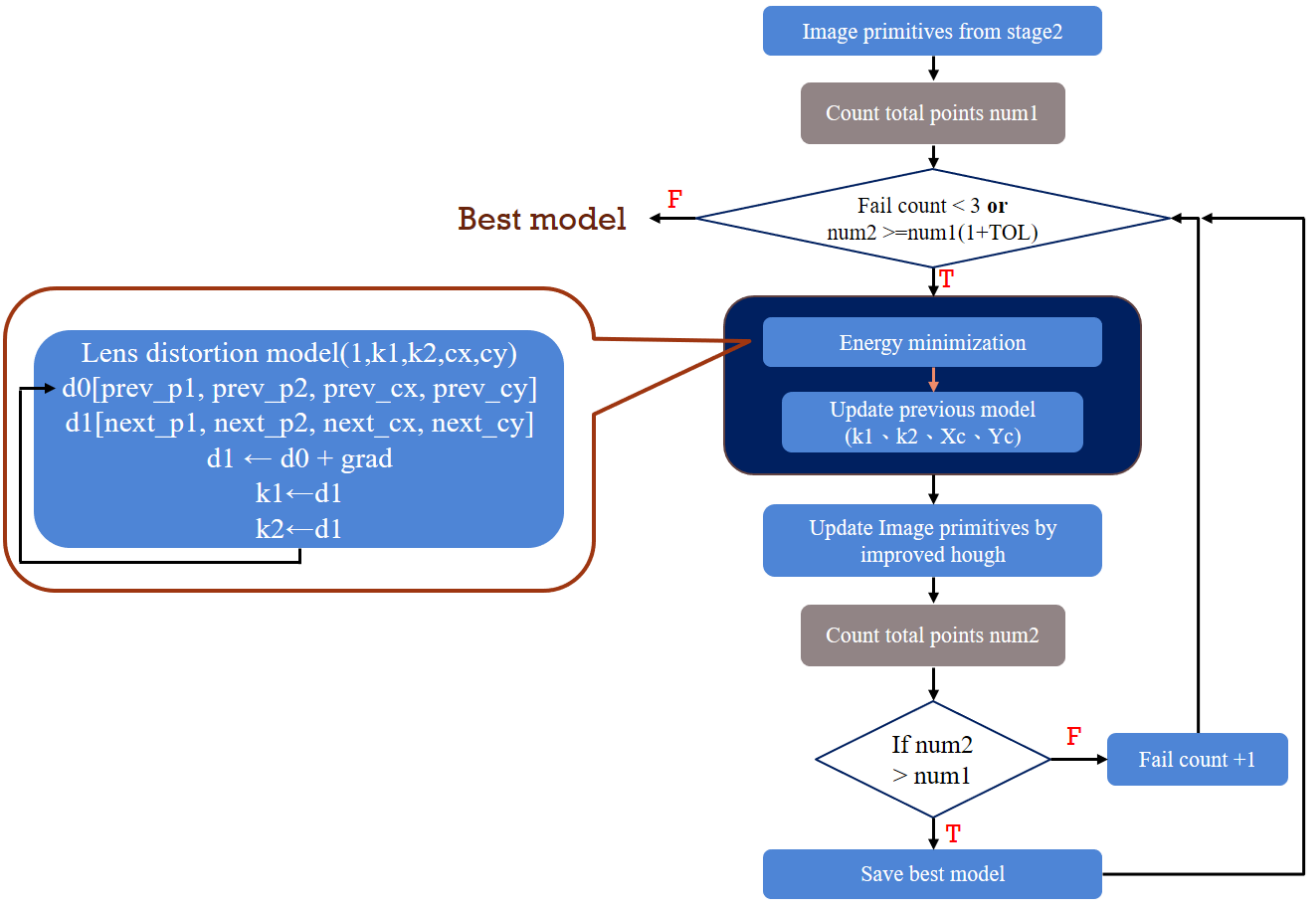

圖 13、Iterative optimization 流程架構圖

使用方法

輸入原始(校正前)的 edge points 資訊與第二階段所得到的校正參數 k1 作為第三階段最佳化的初值。其他參數比照第二階段中的 initial_distortion_parameter, final_distortion_parameter 函數中對應輸入參數的設定值。輸出為最佳化的 loss function 值(不過後續沒有用到)。且本函式執行完後，原輸入的校正參數(i_primitives)會在過程中不斷被更新，最後成為校正模型並做為第四部份影像校正還原的輸入。

5.5.2 total_points

int total_points(image_primitives *ip)

功能描述

對於所輸入的校正線條與對應的 edge points 資訊(image_primitives)，加總每條線所對應的 edge points 以計算校正後的線條中 edge points 數的總和作為校正好壞的衡量指標。

Input

- ip (image_primitives *) : 校正線條與對應的 edge points 資訊，並包含了校正模型。

Output

- 無

Return

- count (int) : 所有校正線條所包含的 edge points 總合。

設計方法

讀取 ip 中的所有線段，累計加總每條線段所包含的 edge points。

使用方法

在第三階段中的校正參數最佳化中，其校正模型的衡量指標與最佳化的收斂條件都以校正後的模型在更新線條與對應的 edge points 後其所包含的 edge points 總數作為校正衡量指標。若校正效果越好，則校正後的線條越直，則所對應到的 edge points 就會越多。

5.5.3 energy_minimization

double energy_minimization(lens_distortion_model *ldm, image_primitives *ip, int w, int h)

功能描述

利用點到對應線條的距離總和作為最佳化的 loss function，對於模型參數(k_1, k_2, C_x, C_y)藉由梯度(gradient)計算與參數更新進行最佳化迭代。

Input

- ldm (lens_distortion_model *) : 校正模型參數 (k_1, k_2, C_x, C_y, type)
- ip (image_primitives *) : 校正線條與對應的 edge points 資訊，並包含了校正模型。
- w (int) : 輸入影像的 width
- h (int) : 輸入影像的 height

Output

- ldm (lens_distortion_model *): 迭代更新後的校正模型參數

Return

- error (double) : 最佳化中多次迭代收斂後的 loss function 值

設計方法

在校正參數最佳化的過程中會分為兩階段:第一階段會只針對 k_1、k_2 進行梯度計算與更新。第二階段會以第一階段所更新的 k_1、k_2 作為初始值再連同校正中心參數 C_x、C_y 一起進行最佳化。如此兩階段最佳化的設計是為在最佳化的過程中會比較偏重於 k_1、k_2 兩參數對於整體最佳化的影響，以避免校正中心參數 C_x、C_y 主導最佳化的過程。此外，為避免校正中心有較大的變化，若更新後的校正中心與校正前有劇烈變化(改變超過原圖長寬的五分之一)，則更新的模型退回到第一階段只考慮 k_1、k_2 影響的更新模型。

使用方法

第三階段中，對於校正模型參數進行最佳化迭代以得到迭代後的更新模型。

5.5.4 model_center_estimation_2p

double model_center_estimation_2p(line_Obj *lines, int lines_size, lens_distortion_model *d, int w, int h, bool v[])

功能描述

此函式為校正模型參數最佳化迭代的主程式。主要針對每個校正參數對校正後的所有 edge points 到對應線條距離的總合作為 loss function 下的梯度計算，並在有效的情況下依據所計算的梯度更新校正模型參數。

Input

- lines (line_Obj *) : 校正線條與對應的 edge points 資訊
- lines_size (int) : 校正線條數目
- d (lens_distortion_model *) : 目前的校正參數
- w (int): 輸入影像的 width
- h (int) : 輸入影像的 height
- v (bool[]) : 決定四個校正參數(k_1, k_2, C_x, C_y)是否在此次迭代中計算其對應梯度並進行更新。本程式中只有兩種設定: 只對 k_1、k_2 進行更新的{True, True, False, False}與四個參數一起更新的{True, True, True, True}。

Output

- d (lens_distortion_model *) : 計算對應參數的梯度並在有效的情況下依據所計算的梯度更新校正模型參數 d。

Return

- minE (double) : 最佳化中多次迭代收斂後的 loss function 值。

設計方法

此函式主要在計算待最佳化的校正參數所對應之梯度，並藉由多次迭代更新後直至收斂。首先，輸入向量 v 決定那些參數需要被更新: 只更新 k_1、k_2 兩參數或是 k_1、k_2、C_x、C_y 全部參數一起更新。如同第二階段中，演算法利用 p_1 與 k_1(以及 p_2 與 k_2)的對應關係公式進行互換。在梯度計算與參數迭代更新上使用 p_1、p_2 以代替 k_1、k_2，而在迭代收斂後再將 p_1、p_2 轉換回 k_1、k_2 值作為更新後的輸出。

Polynomial	Division
$p_1 = k_1 * {r_1}^2 + k_2 * {r_1}^4$ $p_2 = k_1 * {r_2}^2 + k_2 * {r_2}^4$ $k_1 = \frac{p_1 - 16p_2}{-12{r_2}^2}$ $k_2 = \frac{4p_2 - p_1}{-12{r_2}^4}$	$p_1 = \frac{1}{(1 + k_1 * {r_1}^2 + k_2 * {r_1}^4)} - 1$ $p_2 = \frac{1}{(1 + k_1 * {r_2}^2 + k_2 * {r_2}^4)} - 1$ $k_1 = \frac{\frac{-p_1}{1+p_1} + \frac{16p_2}{1+p_2}}{-12{r_2}^2}$ $k_2 = \frac{\frac{-4p_2}{1+p_2} + \frac{p_1}{1+p_1}}{-12{r_2}^4}$

最佳化過程中的 loss function 為校正後的所有 edge points 到對應線條距離的總合來計算梯度，並在有效的情況下依據所計算的梯度更新校正模型參數。程式中的 gradE 向量(4x1)儲存四個校正參數計算後的梯度(gradient)；而 hessianE(4x4)為參數的二階導數矩陣。Ec 結構其前 15 個值儲存著在計算 gradient 與 hessian 時所需預先求得的項(term)，最後一個 entry 儲存著累積的 error 值。最佳化迭代更新的收斂條件為當所有參數(p_1, p_2, C_x, C_y)每次更新的變化量皆未達到更新前的數值的 0.01%並保證至少迭代更新次數至少 100 次。

$$\nabla E(d_0) = \begin{pmatrix} \frac{\partial E(d_0)}{\partial k_1} \\ \frac{\partial E(d_0)}{\partial k_2} \\ \frac{\partial E(d_0)}{\partial x_c} \\ \frac{\partial E(d_0)}{\partial y_c} \end{pmatrix} \qquad \nabla^2 E(d_0) = \begin{pmatrix} \frac{\partial^2 E(d_0)}{\partial k_1^2} & \frac{\partial^2 E(d_0)}{\partial k_1 \partial k_2} & \frac{\partial^2 E(d_0)}{\partial k_1 \partial x_c} & \frac{\partial^2 E(d_0)}{\partial k_1 \partial y_c} \\ \frac{\partial^2 E(d_0)}{\partial k_2 \partial k_1} & \frac{\partial^2 E(d_0)}{\partial k_2^2} & \frac{\partial^2 E(d_0)}{\partial k_2 \partial x_c} & \frac{\partial^2 E(d_0)}{\partial k_2 \partial y_c} \\ \frac{\partial^2 E(d_0)}{\partial x_c \partial k_1} & \frac{\partial^2 E(d_0)}{\partial x_c \partial k_2} & \frac{\partial^2 E(d_0)}{\partial x_c^2} & \frac{\partial^2 E(d_0)}{\partial x_c \partial y_c} \\ \frac{\partial^2 E(d_0)}{\partial y_c \partial k_1} & \frac{\partial^2 E(d_0)}{\partial y_c \partial k_2} & \frac{\partial^2 E(d_0)}{\partial y_c \partial x_c} & \frac{\partial^2 E(d_0)}{\partial y_c^2} \end{pmatrix}$$

$$grad_E(0) \leftarrow -\frac{(E(p_1+h_1,p_2,x_c,y_c)-E(p_1,p_2,x_c,y_c))}{h_1};$$
$$grad_E(1) \leftarrow -\frac{(E(p_1,p_2+h_1,x_c,y_c)-E(p_1,p_2,x_c,y_c))}{h_1};$$
$$grad_E(2) \leftarrow -\frac{(E(p_1,p_2,x_c+h_2,y_c)-E(p_1,p_2,x_c,y_c))}{h_2};$$
$$grad_E(3) \leftarrow -\frac{(E(p_1,p_2,x_c,y_c+h_2)-E(p_1,p_2,x_c,y_c))}{h_2};$$

$$hess_E(0,0) \leftarrow \left(\frac{(E(p_1+h_1,p_2,x_c,y_c)+E(p_1-h_1,p_2,x_c,y_c)-2E(p_1,p_2,x_c,y_c))}{h_1^2}\right);$$
$$hess_E(1,1) \leftarrow \left(\frac{(E(p_1,p_2+h_1,x_c,y_c)+E(p_1,p_2-h_1,x_c,y_c)-2E(p_1,p_2,x_c,y_c))}{h_1^2}\right);$$
$$hess_E(2,2) \leftarrow \left(\frac{(E(p_1,p_2,x_c+h_2,y_c)+E(p_1,p_2,x_c-h_2,y_c)-2E(p_1,p_2,x_c,y_c))}{h_2^2}\right);$$
$$hess_E(3,3) \leftarrow \left(\frac{(E(p_1,p_2,x_c,y_c+h_2)+E(p_1,p_2,x_c,y_c-h_2)-2E(p_1,p_2,x_c,y_c))}{h_2^2}\right);$$

$$hess_E(0,1) \leftarrow hess_E(1,0) \leftarrow \frac{(E(p_1+h_1,p_2+h_1,x_c,y_c)-E(p_1+h_1,p_2,x_c,y_c))-(E(p_1,p_2+h_1,x_c,y_c)-E(p_1,p_2,x_c,y_c))}{h_1^2};$$
$$hess_E(0,2) \leftarrow hess_E(2,0) \leftarrow \frac{(E(p_1+h_1,p_2,x_c+h_2,y_c)-E(p_1+h_1,p_2,x_c,y_c))-(E(p_1,p_2,x_c+h_2,y_c)-E(p_1,p_2,x_c,y_c))}{h_1h_2};$$
$$hess_E(0,3) \leftarrow hess_E(3,0) \leftarrow \frac{(E(p_1+h_1,p_2,x_c,y_c+h_2)-E(p_1+h_1,p_2,x_c,y_c))-(E(p_1,p_2,x_c,y_c+h_2)-E(p_1,p_2,x_c,y_c))}{h_1h_2};$$
$$hess_E(1,2) \leftarrow hess_E(2,1) \leftarrow \frac{(E(p_1,p_2+h_1,x_c+h_2,y_c)-E(p_1,p_2+h_1,x_c,y_c))-(E(p_1,p_2,x_c+h_2,y_c)-E(p_1,p_2,x_c,y_c))}{h_1h_2};$$
$$hess_E(1,3) \leftarrow hess_E(3,1) \leftarrow \frac{(E(p_1,p_2+h_1,x_c,y_c+h_2)-E(p_1,p_2+h_1,x_c,y_c))-(E(p_1,p_2,x_c,y_c+h_2)-E(p_1,p_2,x_c,y_c))}{h_1h_2};$$
$$hess_E(2,3) \leftarrow hess_E(3,2) \leftarrow \frac{(E(p_1,p_2,x_c+h_2,y_c+h_2)-E(p_1,p_2,x_c+h_2,y_c))-(E(p_1,p_2,x_c,y_c+h_2)-E(p_1,p_2,x_c,y_c))}{h_2^2};$$

使用方法

此函數對輸入的當前校正模型根據其校正線條與對應的 edge points 資訊計算校正參數的梯度，最佳化的過程中使用梯度並多次迭代更新模型以趨近最佳的校正參數。

5.5.5 update_rsqmax

double update_rsqmax(Point2D center, int w, int h)

功能描述

計算輸入中心點到影像四個頂點的距離平方，回傳最大的距離平方值

Input

- center (Point2D)：畸變中心點
- w(int)：影像的 width
- h(int)：影像的 height

Output

無

Return

double : 中心點與距中心點最遠角的距離平方

設計方法

將輸入的中心點分別與影像四個頂點做距離平方，並回傳最大值。

使用方法

每次更新校正參數的校正中心後會呼叫函式重新計算校正中心到影像四角的最遠距離平方以更新 p,k 值的互換公式。

5.5.6 E_accumulators_reset

void E_accumulators_reset(E_accumulators *Ec)

功能描述

將 Ec 結構中所有屬性(16 個參數)歸零。

Input

- Ec (E_accumulators *) : Ec 結構，其前 15 個值儲存著在計算 gradient 與 hessian 時所需預先求得的項(term)，最後一個 entry 儲存著累積的 error 值。

Output

Ec (E_accumulators *) : 執行函數後，結構中的屬性皆重置為零。

Return

無

設計方法

結構中的所有屬性歸零。

使用方法

最佳化過程中每次迭代的一開始就會呼叫此函式來重置結構 Ec 中的值，以方便當次迭代的梯度運算。

5.5.7 compute_accumulators

void compute_accumulators(E_accumulators *E, line_Obj *lines, int lines_size, lens_distortion_model ldm, Point2D dc0, const double h1, const double h2, double prevp1, double prevp2, int w, int h)

功能描述

藉由累加計算每條校正線與對應 edge points 的距離總合得到 Ec 中的前 15 個屬性值，以方便之後計算 gradient 向量和 hessian 陣列。

Input

- E (E_accumulators *) : E_accumulators 結構，前 15 個屬性儲存了計算 gradient 與 hessian 所需的加減項。
- lines (line_Obj *) : 當前迭代的校正模型所對應的線條與各線條包含的 edge points 資訊。
- lines_size (int) : 當前迭代的校正模型所對應的校正線條數目。
- ldm (lens_distortion_model) : 當前迭代的校正參數。
- dc0 (Point2D) : 當前迭代的校正中心
- h1 (const double) : 計算 p_1、p_2 導數時所使用的一定值，代表一很小的變化量。
- h2 (const double) : 計算 C_x、C_y 導數時所使用的一定值，代表一很小的變化量。
- prevp1 (double) : 當前迭代的 p_1 值。
- prevp2 (double) : 當前迭代的 p_2 值。
- w (int) : 輸入影像的 width
- h (int) : 輸入影像的 height

Output

E (E_accumulators *) : 函式呼叫後，E_accumulators 結構中的前 15 個屬性值會更新。

Return

無

設計方法

藉由累加計算每條校正線與對應 edge points 的距離總合計算對應輸入的 loss function 值，E_accumulators 結構中的前 15 個屬性儲存了計算 gradient 與 hessian 所需的加減項。

使用方法

每次校正參數的最佳化迭代的過程中，在計算每個參數對應的 gradient 與 hessian 之前會先呼叫此函式，將之後在計算 gradient 與 hessian 中對應輸入的 loss function 值先儲存至 E_accumulators 架構中。

5.5.8 update_den_ks

void update_den_ks(Point2D dc, int w, int h, double *denk1, double *denk2)

功能描述

在每次最佳化迭代更新後，會將更新後的 p 值轉換回 k 值回傳校正參數以便後續更新校正

線條與對應的 edge points。此函式的目的在於計算 p 值轉換回 k 值公式中的分母數值。

Input

- dc (Point2D) : 校正模型中的校正中心座標。
- w (int) : 輸入影像的 width
- h (int) : 輸入影像的 height

Output

- denk1 (double *) : 更新計算 p_1 值轉換回 k_1 值公式中的分母數值。
- denk2 (double *) : 更新計算 p_2 值轉換回 k_2 值公式中的分母數值。

Return

無

設計方法

依照 p 值轉換成 k 值的公式，計算兩參數對應的分母數值。

使用方法

在每次最佳化迭代更新後，會呼叫此函式計算將更新後的 p 值轉換回 k 值公式中分母部分的數值。

5.5.9 update_ldm_and_increment_accumulator

void update_ldm_and_increment_accumulator(lens_distortion_model *ldm, const double n1, const double dk1, const double n2, const double dk2, double *accum, line_Obj *lp)

功能描述

本函式一開始會依照輸入的分子分母數值分別計算對應的 k_1 與 k_2 值，並計算更新後的校正參數所對應的 loss function 值。此處的 loss function 在計算每個校正參數對校正後的所有 edge points 到對應線條距離的總合作為一 error 值。

Input

- ldm (lens_distortion_model *) : 校正模型。
- n1 (const double) : 運算 k_1 的分子部分。
- dk1 (const double) : 運算 k_1 的分母部分。
- n2 (const double) : 運算 k_2 的分子部分。
- dk2 (const double) : 運算 k_2 的分母部分。

- accum (double *) : 依據校正參數累加校正後的 edge points 到對應線條距離的總合。
- lp (line_Obj *) : 某一校正線條與其對應的 edge points 資訊。

Output

- accum (double *) : 依據校正參數累加校正後的 edge points 到對應線條距離的總合，計算對應輸入參數所得到的 error 函數值，以利後續 gradient 與 hessian 的計算。(之後存回對應輸入的 Ec 值中)

Return

無

設計方法

根據函數輸入的分子與分母數值計算對應的 k_1、k_2，在累加計算校正線條與對應 edge points 的距離總合的 error 值(loss function)後儲存累加值至對應輸入的 Ec 屬性中。

使用方法

在計算 Ec 值的函式(compute_accumulators)中，此函數會對於每一條線段與不同輸入參數下所對應的 k_1、k_2 值計算 error 值。迴圈跑完所有校正線條後會得到一累加結果並存入對應的 Ec 屬性以供後續方便計算校正參數的 gradient 與 hessian。

5.5.10 distortion_points_to_line_equation

double distortion_points_to_line_equation(lens_distortion_model *d, line_Obj *Li)

功能描述

函式首先會依輸入的校正模型對線段中的每一點進行校正，校正之後的點再重新計算對應校正線條之方程式。最後再計算點到線的距離總合回傳。

Inpust

- d (lens_distortion_model *) :當前的校正模型
- Li (line_Obj*) :某一線段與對應的 edge points

Output

- Li (line_Obj*) :校正後的點會重新計算出新的線段擬合方程式。

Return

- min (long double) : 回傳校正後點到線段(新擬合的方程式)的距離總合。

設計方法

此函式根據輸入的校正模型校正線段中所包含的點，再使用校正後的點重新計算擬合的線段

方程式。校正更新後的點使用最小距離擬合方法(請詳見 4.5.19)得到新的線段方程式。最後回傳所有校正後的點到更新後線段的距離總合。

使用方法

在計算校正參數的 gradient 與 hessian 數值上需要求得在不同輸入下對應的 loss function 結果(程式中的 error 參數)。其中，loss function 為校正後點到線段的距離總合。此程式函數會在更新點到線的累加距離(update_ldm_and_increment_accumulator)時被呼叫，用於計算依照校正模型計算該線條在校正後的點到線的距離總合。

5.5.11 compute_gradient

void compute_gradient(E_accumulators *E, double *gradE, bool v[], const double h1, const double h2)

功能描述

本函式會依據先前函式所求得的 loss function 結果(儲存在 E 內)計算各校正參數(k_1, k_2, C_x, C_y)的梯度值。

Input

- E (E_accumulators *) : E_accumulators 結構，前 15 個屬性儲存了計算 gradient 與 hessian 所需的加減項。
- v (bool[]) : 一維布林陣列。決定四個校正參數(k_1, k_2, C_x, C_y)是否在此次迭代中計算其對應梯度並進行更新。本程式中只有兩種設定: 只對 k_1、k_2 進行更新的{True, True, False, False}與四個參數一起更新的{True, True, True, True}。
- h1 (const double) : 計算 p_1、p_2 導數時所使用的一定值，代表一很小的變化量。
- h2 (const double) : 計算 C_x、C_y 導數時所使用的一定值，代表一很小的變化量。

Output

- gradE (double *) : 一維 double 陣列，得到每次迭代四個校正參數的梯度值來更新參數。

Return

無

設計方法

依據輸入的一維布林陣列 v[]決定該次迭代是否要針對特定的校正參數進行梯度計算與更新。若 v 為{True, True, False, False}表示只更新 k_1、k_2，而將 C_x、C_y 的梯度值設置為零，表示在此次迭代中不會對於 C_x、C_y 更新；若 v 為{True, True, True, True}則表示四個參數一起更新。

使用方法

對於校正參數最佳化的迭代過程中，演算法藉由計算降低 loss function 的梯度對參數做多次迭代更新已逼近最佳的參數值。

5.5.12 compute_hessian

void compute_hessian(E_accumulators *E, double **hessianE, double gamma, bool v[], const double h1, const double h2)

功能描述

本函式會依據先前函式所求得的 loss function 結果(儲存在 E 內)計算 hessian 陣列中對於各參數組合的二階導數值。

Input

- E (E_accumulators *) : E_accumulators 結構，前 15 個屬性儲存了計算 gradient 與 hessian 所需的加減項。
- gamma (double) : 改變梯度更新速度項。
- v (bool[]) : 一維布林陣列。決定四個校正參數(k_1, k_2, C_x, C_y)是否在此次迭代中計算其對應梯度並進行更新。本程式中只有兩種設定: 只對 k_1、k_2 進行更新的{True, True, False, False}與四個參數一起更新的{True, True, True, True}。
- h1 (const double) : 計算 p_1、p_2 導數時所使用的一定值，代表一很小的變化量。
- h2 (const double) : 計算 C_x、C_y 導數時所使用的一定值，代表一很小的變化量。

Output

- hessianE (double **) : 計算 4x4 參數的二階導數矩陣。

Return

無

設計方法

利用儲存在 E_accumulators 中的屬性依公式計算 hessian 矩陣中的各項 entry。

使用方法

Hessian 矩陣的用途為: (1) 檢驗各校正參數所計算得到的梯度是否有效 (2) 若梯度具有效性，則會在 ami2_gauss 函式中使用 hessian 矩陣的 pivot 值更新梯度值。

5.5.13 ami2_gauss

int ami2_gauss(double **A, double *b, int N)

功能描述

利用計算 hessian 矩陣的高斯消去法檢驗矩陣是否滿秩。若滿秩，則梯度有效(回傳 0)並用 hessian 矩陣中對應的 pivot 值更新梯度值；若不滿秩，則梯度無效(回傳-1)並結束參數最佳化更新的程序。

Input

- A (double **) : 傳入 hessian 矩陣
- b (double *) : 傳入梯度向量
- N (int) : 參數數目

Output

- b (double *) : 更新梯度向量值

Return

- 若 hessian 滿秩，梯度有效且回傳 0。
- 若 hessian 不為滿秩，梯度無效且回傳 -1。

設計方法

對輸入的 hessain 陣列進行高斯消去法檢查梯度是否有效，若有效則更新梯度值並回傳 0。

使用方法

在校正參數最佳化的更新過程中，使用本函式計算 hessian 陣列是否滿秩以檢驗梯度值是否有效於最佳化的參數更新程序。

5.5.14 evaluation

Point2D evaluation(Point2D point, lens_distortion_model model)

功能描述

將輸入的點座標經過輸入的畸變模型得到校正後的點座標

Input

- point (Point2D) : 點座標
- model(lens_distortion_model) : 畸變模型

Output

無

Return

Point2D : 校正後的點座標

設計方法

透過輸入的點座標及畸變模型中心點，算出畸變模型公式中的r^2，再通過函式 polynomial_evaluation 算出 polynomial model($1+k_1*r^2+k_2*r^4$)的值，而 division model 就是 polynomial model 的倒數，最後再用算出的 model 透過以下公式得到校正後的點座標。

$$\begin{pmatrix}\hat{x}-x_c\\ \hat{y}-y_c\end{pmatrix}=L(r)\begin{pmatrix}x-x_c\\ y-y_c\end{pmatrix}$$

$$polynomial:\ \ L(r)=1+k_1*r^2+k_2*r^4$$

$$division:\ \ L(r)=\frac{1}{1+k_1*r^2+k_2*r^4}$$

使用方法

輸入點座標及畸變模型，則可得到校正後的點座標。

5.5.15 polynomial_evaluation

double polynomial_evaluation(double *a, int Na, double x)

功能描述

計算 polynomial model$\left(1+k_1*r^2+k_2*r^4\right)$

Input

- a (double *) : 畸變參數(k1, k2)
- Na(int) : 總共有幾個畸變參數
- x(double) : r^2

Output

無

Return

double : 計算出 polynomial model 的值

設計方法

透過輸入參數的個數，跑迴圈算出($1+k_1*r^2+k_2*r^4$)的值。

使用方法

輸入畸變參數、畸變參數個數以及 r 平方，輸出可得 polynomial model 公式的值。

5.5.16 check_invertibility

bool check_invertibility(lens_distortion_model *ldm, int w, int h)

功能描述

檢驗每次依梯度迭代更新的校正參數是否具一對一的可逆性，以確保在第四階段進行影像還原時可藉由反函數公式反查每點(唯一)對應回的原圖參考像素值。

Input

- ldm (lens_distortion_model *)：當前的校正模型
- w (int)：輸入影像的 width
- h (int)：輸入影像的 height

Output

無

Return

- 可逆：回傳 True
- 不可逆：回傳 False

設計方法

藉由分別對 polynomial 模型與 division 模型的檢驗公式檢驗校正模型的一對一可逆性。

Polynomial Type
$$\begin{cases} 9r_1^4k_1^2 - 20r_1^4k_2 < 0 & if \quad r_1^2k_1 < -\frac{2}{3} \\ 5r_1^4k_2 + 3r_1^2k_1 + 1 > 0 & if \quad r_1^2k_1 \geq -\frac{2}{3}. \end{cases}$$
Division Type
$$\begin{cases} -1 - r_1^2k_1 < r_1^4k_2 < \frac{1-r_1^2k_1}{3} & if \quad -2 < r_1^2k_1 < 2 \\ -1 - r_1^2k_1 < r_1^4k_2 < -\frac{r_1^4k_1^2}{12} & if \quad r_1^2k_1 \geq 2. \end{cases}$$

使用方法

在參數最佳化的迭代過程中，每次迭代會計算參數的梯度值並得到下一輪的更新參數。使用此函式檢視更新後的模型式輔具備一對一可逆性: 若具備可逆性，則更新模型參數並繼續下一輪迭代；若不備可逆性，則忽略此次迭代更新(浪費掉 1 次迭代次數)。

5.5.17 line_equation_distortion_extraction_improved_hough

```
double line_equation_distortion_extraction_improved_hough(
    image_XLD *canny_edges,
    image_primitives *image_primitive,
    float distance_point_line_max,
    int nlines,
    float angle_resolution,
    float distance_resolution,
    float initial_distortion_parameter,
    float final_distortion_parameter,
    float distortion_parameter_resolution,
    float angle_point_orientation_max_difference,
    lens_distortion_model ini_ldm)
```

功能描述

此函數即為第四章所使用函數，詳請見 4.5 指令功能說明。與第四章輸入參不同之處在於參數 initial_distortion_parameter 與 final_distortion_parameter，兩者輸入皆為 0，因此霍夫變換沒有第三維度(沒有迭代 k1)，此函數在這裡的主要目的在於後面針對校正後直線做整併的動作。

6 Image_calibration

6.1 軟體架構

如下圖所示，本函式主要進行影像的校正，透過前面算出的畸變模型參數得到校正影像。

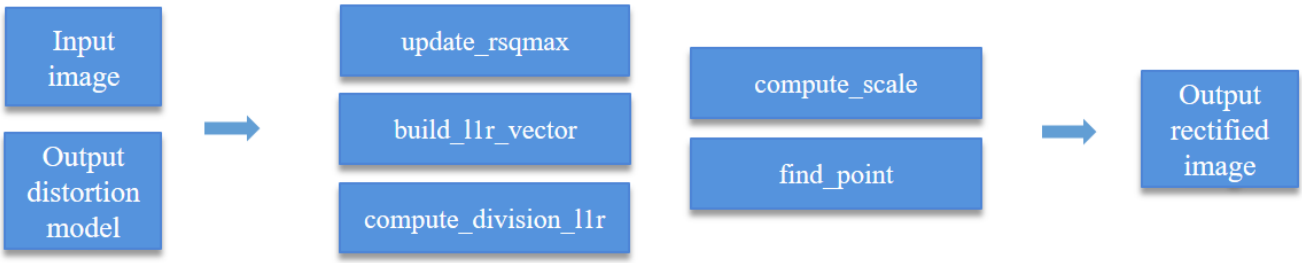

圖 14、Image_calibration 架構圖

6.2 軟體流程

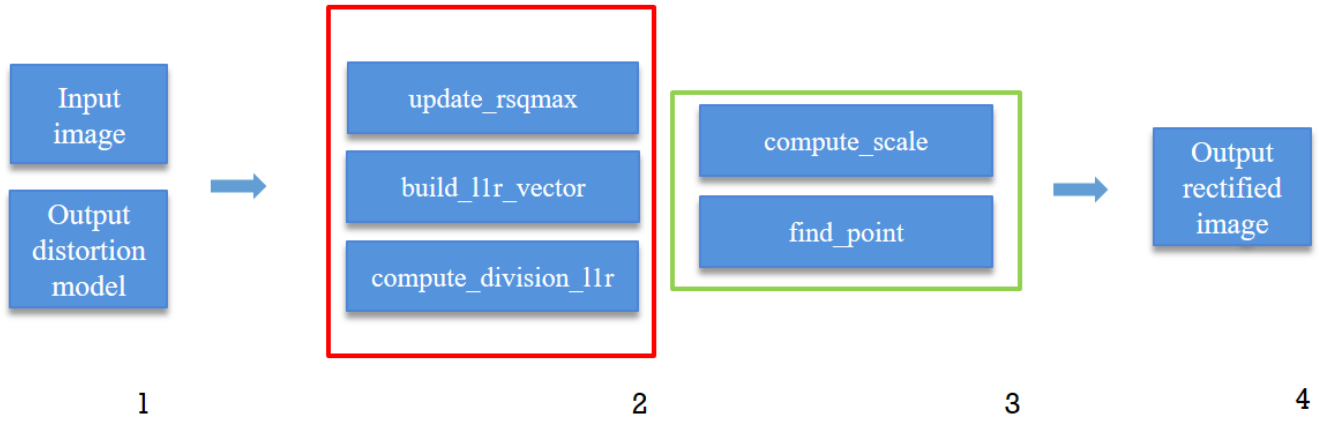

圖 15、Image_calibration 流程圖

1. 讀入畸變影像及畸變模型
2. 計算畸變模型之 inverse function
3. 計算 cropping 相關參數
4. 輸出校正後影像

6.3 資料結構

功能描述

基本資料型態定義

資料結構

```
typedef char                I8;
typedef unsigned char       U8;
typedef short               I16;
typedef unsigned short      U16;
typedef int                 I32;
typedef unsigned int        U32;
typedef long long           I64;
typedef unsigned long long  U64;
typedef float               F32;
typedef double              F64;
```

6.3.1 Point2D

功能描述

存放兩個雙精度浮點數型的座標位置

資料結構

```
typedef struct {
	F64 x;
	F64 y;
}Point2D;
```

6.3.2 lens_distortion_model

功能描述

存放三個畸變參數(1、k1、k2)，畸變中心點以及畸變模型種類(division、polynomial)

資料結構

```
typedef struct {
	double d[3];
	Point2D c;
	int type;
}lens_distortion_model;
```

6.4 指令功能說明

6.4.1 undistort_image_inverse

uchar*** undistort_image_inverse(uchar*** input_image, int* shape, lens_distortion_model *model , bool do_crop, double *fx, double *fy)

功能描述

透過輸入的畸變模型，將 input image(distortion image) 校正成無畸變影像，並截取校正後影像中最大面積的可視部分。

Input

- input_image (uchar***)：輸入畸變影像
- shape (int*)：輸入影像之長、寬、通道數
- model (lens_distortion_model*)：輸入畸變模型
- do_crop (bool)：是否對校正後影像做截取
- fx (double *)：輸入相機內部參數
- fy (double *)：輸入相機內部參數

Output

- shape (int*)：經過 crop 後影像之長、寬、通道數

Return

uchar***： 校正後影像

設計方法

計算出兩個畸變參數與畸變中心點(k_1, k_2, x_c, y_c)後，此函式要透過參數得到校正後影像。首先利用相機模型找到近似的 inverse 模型，本演算法使用牛頓法找 inverse function，接著使用雙線性內插法還原修正後影像。如下圖所示，將畸變的影像A通過相機模型公式得到校正後的影像B，而要得到校正影像必須從校正影像的座標點依序回查畸變影像的座標點，其中需經過相機模型的 inverse distortion model 得到畸變點座標，再利用雙線性內插法計算出像素值並填回校正後影像。

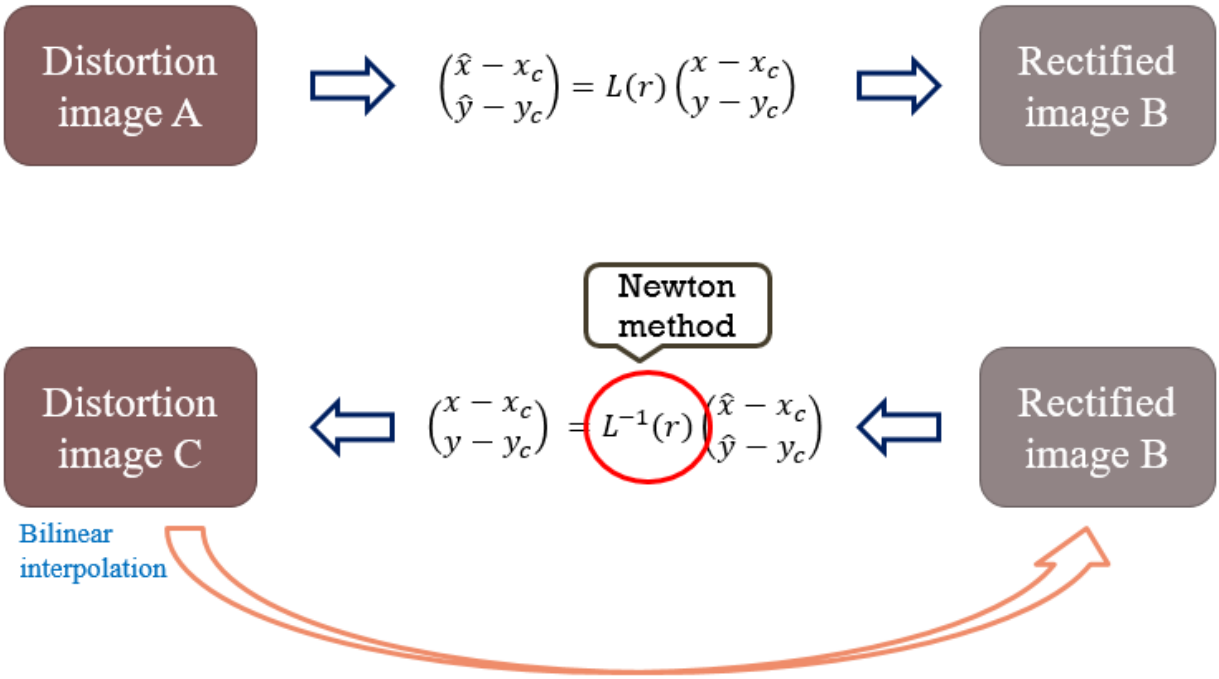

圖 16、獲得校正影像示意圖

使用方法

輸入畸變影像、畸變模型及相應參數，即可得到校正後影像。

6.4.2 update_rsqmax

double update_rsqmax(Point2D center, int w, int h)

功能描述

計算輸入中心點到影像四個頂點的距離平方，回傳最大的距離平方值

Input

- center (Point2D)：畸變中心點
- w(int)：影像的 width
- h(int)：影像的 height

Output

無

Return

double : 中心點與距中心點最遠角的距離平方

設計方法

將輸入的中心點分別與影像四個頂點做距離平方，並回傳最大值。

使用方法

輸入中心點及影像長寬，輸出中心點與距中心點最遠角的距離平方。

6.4.3 build_l1r_vector

int build_l1r_vector(double * invert_r_table, double max_distance_corner, int Na, double *a)

功能描述

計算 polynomial model 的 inverse function

Input

- max_distance_corner (double) : 畸變中心點到影像最遠角之距離
- Na(int) : 2*(畸變參數個數-1)
- a(double *) : 畸變參數根據 r 做升冪排列，缺項補 0，將 r 前的係數存入 a 陣列

Output

- invert_r_table (double *) : 計算出來的 inverse function 在不同 r 之下的值所建立的 table

Return

int : 若回傳-1，則無法計算 inverse function；回傳 1 則計算成功

設計方法

使用牛頓法迭代出 polynomial model 的 inverse function，將 inverse function 在不同 r 之下的值存入 table invert_r_table。

使用方法

輸入 r、畸變參數，則可計算出 inverse function 在不同 r 之下的值，並存入 invert_r_table。

6.4.4 polynomial_evaluation

double polynomial_evaluation(double *a, int Na, double x)

功能描述

計算 polynomial model$\left(1 + k_1 * r^2 + k_2 * r^4\right)$

Input

- a (double *) : 畸變參數(k1, k2)
- Na(int) : 總共有幾個畸變參數
- x(double) : r^2

Output

無

Return

double : 計算出 polynomial model 的值

設計方法

透過輸入參數的個數，跑迴圈算出$(1 + k_1 * r^2 + k_2 * r^4)$的值。

使用方法

輸入畸變參數、畸變參數個數以及 r 平方，輸出可得 polynomial model 公式的值。

6.4.5 compute_division_l1r

static void compute_division_l1r(double * invert_r_table, const double *d, double max_distance_corner)

功能描述

計算 division model 的 inverse function

Input

- d (const double *)：畸變參數(1, k1, k2)
- max_distance_corner(double) : 畸變中心點到影像最遠角之距離

Output

- invert_r_table (double *) : 計算出來的 inverse function 在不同 r 之下的值所建立的 table

設計方法

使用牛頓法迭代出 division model 的 inverse function，將 inverse function 在不同 r 之下的值存入 table invert_r_table。

使用方法

輸入 r、畸變參數，則可計算出 inverse function 在不同 r 之下的值，並存入 invert_r_table。

6.4.6 compute_scale

static double compute_scale(int width, int height, lens_distortion_model *ldm, double *fx, double *fy)

功能描述

計算在回查畸變影像座標點時乘上的 scale 參數，以及 fx、fy

Input

- width (int)：輸入影像之寬
- height (int)：輸入影像之高
- ldm(lens_distortion_model *)：畸變模型

Output

- fx(double *)：相機內部參數
- fy(double *)：相機內部參數

Return

double : 計算出的 scale

設計方法

在回查座標點時，座標會乘上一個 scale 參數，這會對顯示出的校正後影像造成一個縮放的影響，若此時座標超出影像範圍，意即無法查到像素值，則此時校正影像上填回的像素值為 0，會產生部分黑色未定義點的區域。而在得到校正影像後我們會對影像作擷取的動作，把未定義點的部分切掉，保留有定義點最大面積的影像。scale 的運算方法如下式，式中$center$代表畸變中心點，即為(C_x, C_y)，$t1$、$t2$、$t3$、$t4$代表原始畸變影像四個頂點校正後的點座標。乘上 scale 主要在呈現如下圖之影像，原圖所有的像素點都包含在校正後影像中。

$$scale = max\left\{\frac{|(t1\ - center)|}{|center|}, \frac{|(t2\ - center)|}{|center|}, \frac{|(t3\ - center)|}{|center|}, \frac{|(t4\ - center)|}{|center|}\right\}$$

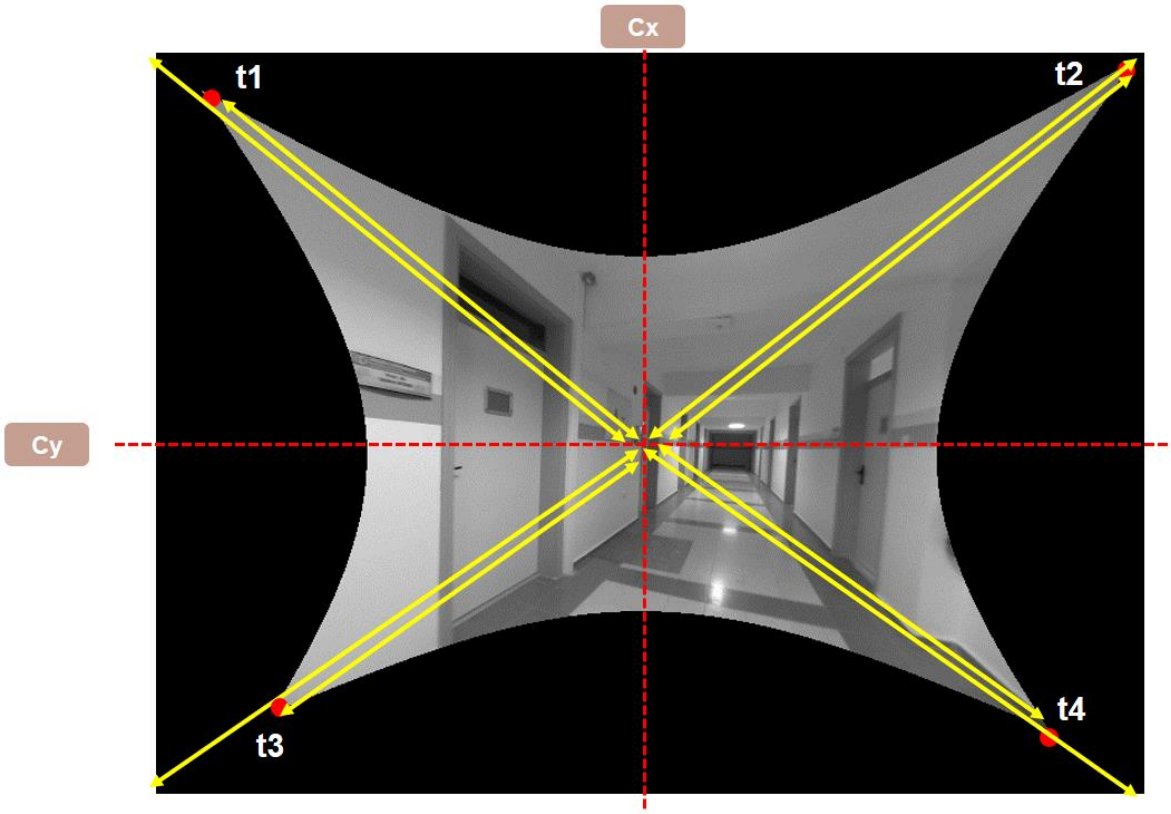

圖 17、參數 scale 運算示意圖

而在 HALCON 中的f_x、f_y參數，我們利用類似的概念來計算，如下式：

$$f_x = min\left\{\frac{|(w1\ - center)|}{|center.x|}, \frac{|(w2\ - center)|}{|center.x|}\right\}$$

$$f_y = min\left\{\frac{|(h1\ - center)|}{|center.y|}, \frac{|(h2\ - center)|}{|center.y|}\right\}$$

其中 $w1$、$w2$、$h1$、$h2$ 分別代表 $(0, center.y)$、$(width, center.y)$、$(center.x, 0)$、$(center.x, height)$經過校正後的點座標，$center$代表畸變中心點。

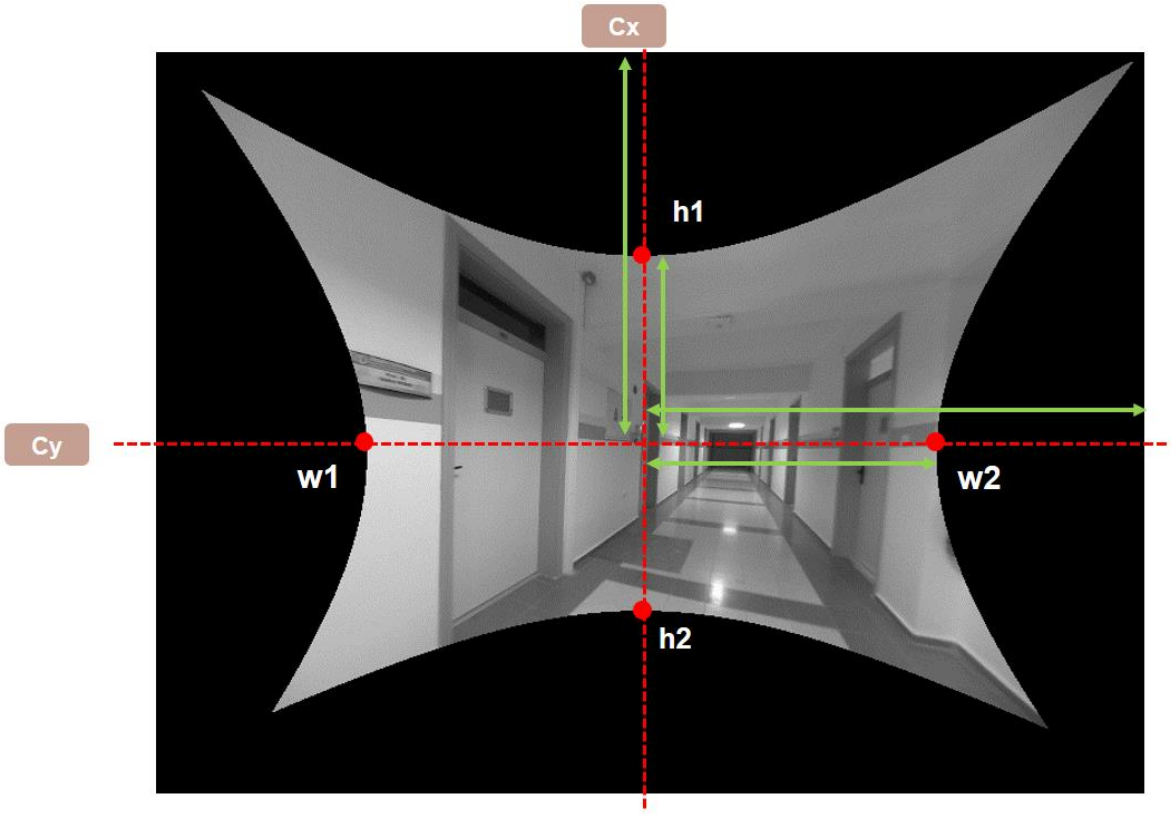

圖 18、參數f_x、f_y運算示意圖

使用方法

輸入影像長寬及畸變模型，輸出參數 scale 及 fx、fy。

6.4.7 evaluation

Point2D evaluation(Point2D point, lens_distortion_model model)

功能描述

將輸入的點座標經過輸入的畸變模型得到校正後的點座標

Input

- point (Point2D)：點座標
- model(lens_distortion_model)：畸變模型

Output

無

Return

Point2D：校正後的點座標

設計方法

透過輸入的點座標及畸變模型中心點，算出畸變模型公式中的r^2，再通過函式 polynomial_evaluation 算出 polynomial model($1+k_1*r^2+k_2*r^4$)的值，而 division model 就是 polynomial model 的倒數，最後再用算出的 model 透過以下公式得到校正後的點座標。

$$\begin{pmatrix}\hat{x}-x_c\\ \hat{y}-y_c\end{pmatrix}=L(r)\begin{pmatrix}x-x_c\\ y-y_c\end{pmatrix}$$

$$polynomial:\ \ L(r)=1+k_1*r^2+k_2*r^4$$

$$division:\ \ L(r)=\frac{1}{1+k_1*r^2+k_2*r^4}$$

使用方法

輸入點座標及畸變模型，則可得到校正後的點座標。

6.4.8 multiply

Point2D multiply(Point2D a, double b)

功能描述

輸入座標點與輸入常數做相乘

Input

- a (Point2D)：輸入點座標
- b (double)：輸入常數

Output

無

Return

Point2D：輸入座標點與輸入常數相乘後的座標點

設計方法

根據以下點座標 a 與常數 c 相乘公式作運算：

$$b=a(a_x,a_y)*\mathrm{c}=(c*a_x,c*a_y)$$

回傳相乘後之 b 點座標。

使用方法

輸入欲相乘之座標點與常數。

6.4.9 substract

Point2D substract(Point2D a, Point2D b)

功能描述

將兩個輸入點座標作相減

Input

- a(Point2D)：輸入點座標
- b(Point2D)：輸入點座標

Output

無

Return

Point2D：回傳 a、b 兩點相減後的點座標

設計方法

輸入兩點 a (a_x, a_y)、b (b_x, b_y)，做下列相減運算：

$$c = a - b = (a_x - b_x, a_y - b_y)。$$

回傳相減後之 c 點座標。

使用方法

輸入兩個點座標，輸出相減後的點座標。

6.4.10 norm

double norm(Point2D k)

功能描述

計算輸入點座標與原點的歐式距離

Input

- k(Point2D)：輸入點座標

Output

無

Return

double：回傳輸入點座標與原點之歐式距離值

設計方法

以下方公式計算輸入點座標與原點之歐式距離，並回傳其結果。

$$\|(x,y)\| = \sqrt{x^2+y^2}$$

使用方法

輸入一點，輸出此點與原點之歐式距離。

6.4.11 find_point

int find_point(uchar** imbw, int* shape, int direction)

功能描述

此函式主要用於校正後影像在尋找截圖範圍的上下左右界。

Input

- imbw (uchar**) : 偵測後的黑白影像。
- shape (int*) : 影像維度大小 (高/寬/頻道數)。
- direction (int) : 回傳的截圖範圍種類 (上界:0, 下屆:1, 左界:2, 右界:3)

Output

無

Return

- bound (int) : 若方向為 0，則回傳截圖之上界(對最大連通物件找尋最大 row 值)
  若方向為 1，則回傳截圖之下界(對最大連通物件找尋最小 row 值)
  若方向為 2，則回傳截圖之左界(對最大連通物件找尋最大 col 值)
  若方向為 3，則回傳截圖之右界(對最大連通物件找尋最小 col 值)

設計方法

藉由 check_map 標出為定義的區域後，找出有定義與無定義的標界區域並分出上界線、下界線、左界線與右界線。對四種界線作 8 連通演算法尋找最大連通元件。針對上界線連通元件找其下界、下界線連通元件找其上界、左界線連通元件找其右界、右界線連通元件找其左界來定義截圖的四個邊界。

使用方法

在完成影像的校正還原後，呼叫此函式來找排除未定義的區域並截圖作為作後輸出結果。

7 參考文獻

[1] M. Alem_an-Flores, L. Alvarez, L. Gomez, and D. Santana-Cedr_es, Automatic Lens Distortion Correction Using One-Parameter Division Models, Image Processing On Line, 4(2014), pp. 327 - 343.

[2] M. Alem_an-Flores, L. Alvarez, L. Gomez, and D. Santana-Cedr_es, Line detection in images showing signi_cant lens distortion and application to distortion correction, Pattern Recognition Letters, 36 (2014), pp. 261 - 271.

[3] T. Thormälen, H. Broszio: “Automatic line-based estimation of radial lens distortion”; in: Integrated Computer-Aided Engineering; vol. 12; pp. 177-190; 2005.

[4] Jos´e Henrique Brito: “Radial Distortion Self-Calibration” ;in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition;2013

[5] Benligiray, B.; Topal, C., "Blind rectification of radial distortion by line straightness," Proc. Eur. Signal Process. Conf. (EUSIPCO), 2016 Chernov, N. (2010). Circular and Linear Regression Fitting Circles and Lines by Least Squares. CRC Press.

[6] Cuneyt Akinlar and Cihan Topal: “EDLINES: REAL-TIME LINE SEGMENT DETECTION BY EDGE DRAWING (ED) ”; in: 18th IEEE International Conference on Image Processing;2011 GSL-reference

[7] Benligiray, B.; Topal, C., "Lens distortion rectification using triangulation based interpolation," International Symposium on Visual Computing (ISVC), 2015

[8] Daniel Santana-Cedrés, Luis Gómez, Miguel Alemán-Flores, Agustín Salgado, Julio Esclarín, Luis Mazorra, and Luis Álvarez, An Iterative Optimization Algorithm for Lens Distortion Correction Using Two-Parameter Models, Image Processing On Line, 6 (2016), pp. 326–364

[9] Alvarez, Luis, and Luis Mazorra. "Signal and image restoration using shock filters and anisotropic diffusion." SIAM journal on numerical analysis 31.2 (1994): 590-605.